ECC (Everything Claude Code) — قابلیت‌های نصب‌شده روی OpenCode

نسخهٔ ECC: 2.2.3 | منبع: affaan-m/ECC (fork: javadhamed/ECC) | تاریخ تولید: 20-10-2026 | توضیحات به زبان فارسی

این سند فهرست کامل همهٔ hooks، plugins، tools، skills، commands، agents و MCP servers نصب‌شده از ECC روی OpenCode را با توضیحات فارسی و روش استفاده ارائه می‌دهد.

فهرست مطالب:

۱. خلاصهٔ آماری
۲. Agents (26)
۳. Commands (35 ECC)
۴. Skills (354)
۵. Tools (9)
۶. Plugins (4)
۷. Hooks (27 در 8 گروه)
۸. MCP Servers (34 تعریف‌شده)
۹. روش استفادهٔ کلی

۱. خلاصهٔ آماری

بخش	تعداد	توضیح
Agents	26	زیر-عامل‌های تخصصی برای delegation
Commands (ECC)	35	دستورات slash در OpenCode
Skills	354	مهارت‌های بارگذاری‌شده (شامل غیر-ECC)
Tools	9	ابزارهای سفارشی OpenCode
Plugins	4	پلاگین‌های TypeScript
Hooks	27	در 8 گروه رویداد
MCP Servers	34	تعریف‌شده در mcp-servers.json (نیاز به فعال‌سازی دستی)

۲. Agents (26)

زیر-عامل‌های تخصصی که در `/config/opencode/prompts/agents./~` قرار دارند. در OpenCode با `agent-name@` یا از طریق Task tool فراخوانی می‌شوند.

#	Agent	توضیحات کامل (فارسی)
1	`architect`	معمار نرم‌افزار ارشد متخصص در طراحی سیستم‌های مقیاس‌پذیر و قابل نگهداری. الگوها، مرزهای ماژول، trade-offها، مقیاس‌پذیری و تصمیمات معماری را بررسی می‌کند. برای تصمیمات معماری، طراحی سیستم و انتخاب تکنولوژی استفاده کنید.
2	`build-error-resolver`	متخصص رفع خطاهای build با تغییرات حداقلی و جراحی. خطاهای کامپایل، مشکلات پیکربندی و وابستگی‌ها را با کمترین تغییر ممکن حل می‌کند. وقتی build شکسته است و نیاز به رفع سریع دارید استفاده کنید.
3	`code-reviewer`	بازبین کد ارشد که کیفیت، امنیت و قابلیت نگهداری را تضمین می‌کند. کیفیت کد، امنیت، عملکرد و بهترین شیوه‌ها را بررسی می‌کند. پس از نوشتن یا تغییر کد برای بازبینی استفاده کنید.
4	`cpp-build-resolver`	متخصص رفع خطاهای build C++ شامل CMake و linker. خطاهای کامپایل C++، مشکلات CMake و هشدارهای linker را با تغییرات حداقلی حل می‌کند.
5	`cpp-reviewer`	بازبین کد C++ ارشد که استانداردهای C++ مدرن و بهترین شیوه‌ها را تضمین می‌کند. الگوهای مدرن C++، ایمنی حافظه و عملکرد را بررسی می‌کند.
6	`database-reviewer`	متخصص بهینه‌سازی پایگاه داده. طراحی schema، کوئری‌ها، ایندکس‌ها و عملکرد DB را بررسی می‌کند. برای مشکلات پایگاه داده، بهینه‌سازی کوئری و طراحی schema استفاده کنید.
7	`doc-updater`	متخصص مستندسازی و codemap. مستندات، codemapها و README را به‌روزرسانی می‌کند. پس از تغییرات کد برای به‌روزرسانی مستندات استفاده کنید.
8	`docs-lookup`	متخصص مستندات که با استفاده از Context7 MCP (نه داده‌های آموزش) به سؤالات دربارهٔ کتابخانه‌ها، فریمورک‌ها و APIها پاسخ می‌دهد. برای جستجوی مستندات به‌روز کتابخانه‌ها استفاده کنید.
9	`e2e-runner`	اجرای تست‌های E2E با Playwright. تست‌های end-to-end را تولید و اجرا می‌کند. برای تست‌های E2E و سناریوهای کاربر استفاده کنید.
10	`go-build-resolver`	متخصص رفع خطاهای build Go. خطاهای کامپایل Go، مشکلات vet و وابستگی‌ها را حل می‌کند.
11	`go-reviewer`	بازبین کد Go ارشد که الگوهای ایدیوماتیک Go و بهترین شیوه‌ها را تضمین می‌کند. برای بازبینی کد Go استفاده کنید.
12	`harness-optimizer`	بهینه‌ساز harness که کیفیت، قابلیت اطمینان و کارایی harness عامل را بهبود می‌دهد. پیکربندی agent، tool و hook را بهینه می‌کند.
13	`java-build-resolver`	متخصص رفع خطاهای build Java/Maven/Gradle. خطاهای کامپایل Java، مشکلات Maven/Gradle و وابستگی‌ها را حل می‌کند.
14	`java-reviewer`	مهندس Java ارشد که استانداردهای Java ایدیوماتیک، Spring Boot و Quarkus را تضمین می‌کند.
15	`kotlin-build-resolver`	متخصص رفع خطاهای build Kotlin/Gradle. خطاهای کامپایل Kotlin و مشکلات Gradle را حل می‌کند.
16	`kotlin-reviewer`	بازبین کد Kotlin و Android/KMP ارشد که کد ایدیوماتیک، ایمن و قابل نگهداری را تضمین می‌کند.
17	`loop-operator`	اپراتور حلقهٔ agent که حلقه‌های agent را مدیریت و کنترل می‌کند. نظارت بر اجرای agent loops، کنترل و بازیابی.
18	`php-reviewer`	بازبین کد PHP ارشد که استانداردهای PHP و بهترین شیوه‌ها را تضمین می‌کند.

19	`planner`	متخصص برنامه‌ریزی پیاده‌سازی. درخواست‌ها را تحلیل و طرح‌های جامع و قابل اجرا ایجاد می‌کند. ویژگی‌ها را به مراحل قابل مدیریت تقسیم می‌کند، وابستگی‌ها و ریسک‌ها را شناسایی می‌کند. قبل از پیاده‌سازی ویژگی‌های پیچیده استفاده کنید.
20	`python-reviewer`	بازبین کد Python ارشد که استانداردهای Pythonic و بهترین شیوه‌ها را تضمین می‌کند.
21	`refactor-cleaner`	پاک‌سازی کد مرده و تجمیع تکراری‌ها. کد استفاده‌نشده، importهای بلااستفاده و تکراری‌ها را شناسایی و حذف می‌کند.
22	`ruby-reviewer`	بازبین کد Ruby و Rails ارشد که استانداردهای Ruby و بهترین شیوه‌ها را تضمین می‌کند.
23	`rust-build-resolver`	متخصص رفع خطاهای build Rust و borrow checker. خطاهای کامپایل Rust و مشکلات ownership را حل می‌کند.
24	`rust-reviewer`	بازبین کد Rust ارشد که ایمنی، الگوهای ایدیوماتیک و عملکرد را تضمین می‌کند.
25	`security-reviewer`	بازبین امنیتی که آسیب‌پذیری‌ها، مشکلات امنیتی و ریسک‌ها را شناسایی می‌کند. OWASP Top 10، injection، auth، secrets و پیکربندی ناامن را بررسی می‌کند.
26	`tdd-guide`	متخصص TDD که توسعهٔ test-first با پوشش جامع را تضمین می‌کند. تست‌ها را قبل از کد می‌نویسد و چرخهٔ red-green-refactor را دنبال می‌کند.

۳. Commands (35 دستور ECC)

دستورات slash ECC در `/config/opencode/commands./~`. استفاده: `command-name [args]/`

#	Command	توضیحات کامل (فارسی)
1	`build-fix/`	رفع خطاهای build و TypeScript با تغییرات حداقلی. خطاهای کامپایل، type errors و مشکلات build را حل می‌کند.
2	`checkpoint/`	ذخیرهٔ وضعیت تأیید و checkpoint پیشرفت. برای ذخیرهٔ پیشرفت کار و امکان بازگشت استفاده کنید.
3	`code-review/`	بازبینی کد برای کیفیت، امنیت و قابلیت نگهداری. کد تغییریافته را بررسی می‌کند.
4	`e2e/`	تولید و اجرای تست‌های E2E با Playwright. برای تست‌های end-to-end استفاده کنید.
5	`eval/`	اجرای ارزیابی در برابر معیارهای پذیرش. نتایج را با معیارها مقایسه می‌کند.
6	`evolve/`	تحلیل instincts و پیشنهاد یا تولید ساختارهای تکامل‌یافته. الگوهای یادگرفته‌شده را به skills/commands تبدیل می‌کند.
7	`go-build/`	رفع خطاهای build و go vet. مشکلات کامپایل Go را حل می‌کند.
8	`go-review/`	بازبینی کد Go برای الگوهای ایدیوماتیک. استانداردهای Go را بررسی می‌کند.
9	`go-test/`	گردش‌کار TDD Go با تست‌های table-driven. TDD برای Go.
10	`harness-audit/`	اجرای audit قطعی harness مخزن و بازگرداندن scorecard اولویت‌بندی‌شده. سلامت harness را ارزیابی می‌کند.
11	`instinct-export/`	صادرات instincts برای اشتراک‌گذاری. instincts یادگرفته‌شده را صادر می‌کند.
12	`instinct-import/`	وارد کردن instincts از منابع خارجی. instincts را از فایل وارد می‌کند.
13	`instinct-status/`	نمایش instincts یادگرفته‌شده (پروژه + global) با confidence. وضعیت یادگیری را نشان می‌دهد.

14	`learn/`	استخراج الگوها و یادگیری‌ها از نشست فعلی. دانش را از session استخراج می‌کند.
15	`loop-start/`	شروع حلقهٔ agent. یک agent loop جدید را آغاز می‌کند.
16	`loop-status/`	وضعیت حلقهٔ agent. وضعیت حلقهٔ در حال اجرا را نشان می‌دهد.
17	`model-route/`	مسیریابی model. model مناسب برای task را انتخاب می‌کند.
18	`orchestrate/`	هماهنگی چند agent برای taskهای پیچیده. چند agent را به‌صورت موازی هماهنگ می‌کند.
19	`plan/`	ایجاد طرح پیاده‌سازی با ارزیابی ریسک. طرح جامع برای ویژگی ایجاد می‌کند.
20	`projects/`	فهرست پروژه‌های ثبت‌شده و تعداد instincts. پروژه‌ها را نشان می‌دهد.
21	`promote/`	ارتقای instincts پروژه به scope global. instincts را global می‌کند.
22	`quality-gate/`	دروازهٔ کیفیت. بررسی می‌کند آیا کد آمادهٔ merge است.
23	`refactor-clean/`	حذف کد مرده و تجمیع تکراری‌ها. کد استفاده‌نشده را پاک می‌کند.
24	`rust-build/`	رفع خطاهای build Rust و borrow checker. مشکلات Rust را حل می‌کند.
25	`rust-review/`	بازبینی کد Rust برای ownership، ایمنی و الگوهای ایدیوماتیک.
26	`rust-test/`	گردش‌کار TDD Rust با تست‌های unit و property.
27	`security/`	اجرای بازبینی امنیتی جامع. آسیب‌پذیری‌ها را شناسایی می‌کند.
28	`security-scan/`	اجرای AgentShield روی agent، hook، MCP، permission و secret surfaces.
29	`setup-pm/`	پیکربندی ترجیح package manager. npm/pnpm/yarn/bun را تنظیم می‌کند.
30	`skill-create/`	تولید skills از تحلیل git history. skills جدید از تاریخچهٔ git ایجاد می‌کند.
31	`tdd/`	اعمال گردش‌کار TDD با پوشش ۸۰٪+. توسعهٔ test-first را اعمال می‌کند.
32	`test-coverage/`	تحلیل و بهبود پوشش تست. فایل‌های کم‌پوشش را شناسایی می‌کند.
33	`update-codemaps/`	به‌روزرسانی codemapها برای ناوبری codebase. نقشهٔ کد را به‌روز می‌کند.
34	`update-docs/`	به‌روزرسانی مستندات برای تغییرات اخیر. مستندات را به‌روز می‌کند.
35	`verify/`	اجرای حلقهٔ تأیید برای اعتبارسنجی پیاده‌سازی. build، typecheck، lint، تست و security را بررسی می‌کند.

۴. Skills (354)

مهارت‌های بارگذاری‌شده در `/config/opencode/skills./~`. هر skill یک پوشه با فایل `SKILL.md` است. بارگذاری با ابزار `skill` یا به‌صورت خودکار توسط agent.

#	Skill	توضیحات کامل (فارسی)
1	`accessibility`	طراحی، پیاده‌سازی و audit UI قابل دسترس بر اساس WCAG 2.2 Level AA در Web، iOS و Android. شامل semantic ARIA roles و labels، accessibility traits، hints، مدیریت focus، contrast، target size و پشتیبانی screen-reader. هنگام ساخت یا audit UI برای keyboard navigation، accessibility compliance یا screen-reader support استفاده کنید.

2	`agent-architecture-audit`	تشخیص full-stack برای اپلیکیشن‌های agent و ۱۲ stack .LLM-لایهٔ agent را برای wrapper regression، memory pollution، tool discipline failures، hidden repair loops و rendering corruption audit می‌کند. یافته‌های severity-ranked با code-first fixes تولید می‌کند. برای توسعه‌دهندگان agent applications، autonomous loops یا هر feature مبتنی بر LLM ضروری است. هنگامی که agent یا LLM feature misbehave می‌کند و layer شکست‌خورده ناشناخته است یا قبل از shipping agent stack استفاده کنید.
3	`agent-eval`	مقایسهٔ head-to-head coding agentها (Claude Code، Aider، Codex و...) روی taskهای سفارشی با متریک‌های pass rate، cost، time و consistency. هنگام انتخاب بین coding agentها یا وقتی تغییری در setup agent نیاز به pass rate، cost و time اندازه‌گیری‌شده دارد (نه impression) استفاده کنید.
4	`agent-harness-construction`	طراحی و بهینه‌سازی action spaces، tool definitions و observation formatting agent AI برای completion rate بالاتر. هنگام تعریف یا بازنگری tool set، action space یا observation format agent استفاده کنید.
5	`agent-introspection-debugging`	گردش‌کار self-debugging ساختاریافته برای شکست‌های agent AI با استفاده از capture، diagnosis، contained recovery و introspection reports. هنگامی که یک agent run شکست می‌خورد و نیاز به diagnosis قابل تکرار (نه retry) دارید استفاده کنید.
6	`agent-payment-x402`	افزودن x402 payment execution به agentهای AI با per-task budgets، spending controls و non-custodial wallets. پشتیبانی از Solana، X Layer (OKX Payments)، Base (agentwallet-sdk) و multi-network EVM. هنگامی که agent باید خودش پرداخت کند و نیاز به per-task budgets، spending controls و non-custodial wallet دارد استفاده کنید.
7	`agent-runtime-gateway-smoke-test`	تأیید یک agent API محلی، temporary gateway tunnel و remote sandbox callback با یک task بدون tool، سپس بازگرداندن اتصال اصلی app.
8	`agent-security-hardening`	راهنمای security hardening برای frameworkهای agent AI که content نامعتبر پردازش می‌کنند، tool invoke می‌کنند، فایل workspace می‌نویسند، runtime identifiers مدیریت می‌کنند یا credentials handle می‌کنند. هنگام ساخت یا بازبینی agent runtime، autonomous worker، tool gateway، memory service یا multi-tenant agent deployment استفاده کنید.
9	`agent-self-evaluation`	پس از تکمیل هر task غیرtrivial استفاده کنید. agent خروجی خود را روی ۵ محور (accuracy، completeness، clarity، actionability، conciseness) با evidence مشخص per criterion امتیازدهی می‌کند. scorecard ساختاریافتهٔ ۱-۵ با پیشنهادات بهبود تولید می‌کند.
10	`agent-sort`	ساخت ECC install plan مبتنی بر evidence برای یک repo خاص با hooks، rules، commands، sorting skills و extras به buckets DAILY vs LIBRARY با استفاده از parallel repo-aware review passes. هنگامی که ECC باید به چیزی که پروژه واقعاً نیاز دارد trim شود (نه full bundle) استفاده کنید.
11	`agentic-engineering`	عمل به‌عنوان agentic engineer با decomposition، eval-first execution و cost-aware model routing. هنگام planning یا اجرای engineering work که agentها end-to-end انجام می‌دهند استفاده کنید.
12	`agentic-os`	ساخت operating systemهای multi-agent پایدار روی Claude Code. kernel architecture، specialist agents، slash commands، file-based memory، scheduled automation و state management بدون database خارجی. هنگام ساخت multi-agent system پایدار با memory، commands و scheduling خودش استفاده کنید.
13	`ai-first-engineering`	مدل operating engineering برای تیم‌هایی که AI agentها سهم بزرگی از implementation output تولید می‌کنند. هنگام تنظیم process تیم، review gates یا ownership rules برای codebase که بیشتر توسط agentها نوشته شده استفاده کنید.
14	`ai-regression-testing`	استراتژی‌های regression testing برای توسعهٔ Sandbox-mode API testing. AI-assisted بدون database dependencies، automated bug-check workflows و patterns برای گرفتن AI blind spots (که همان model می‌نویسد و review می‌کند). هنگام افزودن regression coverage به کد AI-assisted یا وقتی همان model هم نوشته هم review کرده استفاده کنید.

15	`android-clean-architecture`	الگوهای Clean Architecture برای پروژه‌های Android و Kotlin Multiplatform — ساختار module، dependency rules، UseCases، Repositories و data layer patterns. هنگام ساختاردهی modules، layers یا data flow در Android/KMP استفاده کنید.
16	`angular-developer`	تولید کد Angular و راهنمای معماری. trigger هنگام ایجاد components، projects یا services، یا best practices روی reactivity (signals، linkedSignal، resource)، forms، DI، routing، SSR، accessibility (ARIA)، animations، styling (component styles، Tailwind CSS)، testing یا CLI tooling.
17	`api-connector-builder`	ساخت یک API connector یا provider جدید با match دقیق pattern یکپارچه‌سازی موجود در repo. هنگام افزودن یک integration دیگر بدون اختراع architecture دوم استفاده کنید.
18	`api-design`	الگوهای طراحی REST API شامل resource naming، status codes، pagination، filtering، error responses، versioning و rate limiting برای APIهای production. هنگام طراحی یا بازبینی REST endpoints، resource names، status codes، pagination یا versioning استفاده کنید.
19	`architecture-decision-records`	ثبت تصمیمات معماری به‌صورت فایل‌های ADR شماره‌گذاری‌شده در /docs/adr با context، alternatives considered، consequences و index README. هنگامی که کاربر می‌گوید 'record this decision' یا 'ADR this'، بین frameworkها یا databaseها انتخاب می‌کند، trade-offها را بحث می‌کند یا می‌پرسد چرا codebase این شکل است استفاده کنید.
20	`article-writing`	نوشتن مقالات، guideها، blog postها، tutorialها، newsletter issueها و سایر محتوای long-form در یک voice متمایز derived از examples یا brand guidance. هنگامی که کاربر محتوای نوشته‌شدهٔ polished بیش از یک پاراگراف می‌خواهد، به‌ویژه وقتی voice consistency، structure و credibility مهم است استفاده کنید.
21	`automation-audit-ops`	گردش‌کار audit inventory و overlap automation مبتنی بر evidence برای ECC. هنگامی که کاربر می‌خواهد بداند کدام MCP servers، connectors، hooks، jobs یا wrappers زنده، شکسته، redundant یا missing هستند قبل از fix کردن استفاده کنید.
22	`autonomous-agent-harness`	تبدیل Claude Code به یک agent system کاملاً autonomous با persistent memory، scheduled operations، computer use و task queuing. جایگزین standalone agent frameworks (Hermes، AutoGPT) با استفاده از native MCP tools، dispatch، crons و memory Claude Code. هنگامی که کاربر continuous autonomous operation، scheduled tasks یا self-directing agent loop می‌خواهد استفاده کنید.
23	`autonomous-loops`	الگوها و معماری‌های حلقه‌های autonomous Claude Code — از simple sequential pipelines تا RFC-driven multi-agent DAG systems. فقط برای compatibility نگه داشته شده؛ برای راهنمایی جدید از continuous-agent-loop استفاده کنید.
24	`backend-patterns`	الگوهای معماری backend، طراحی API، بهینه‌سازی database و best practices server-side برای Node.js، Express و Next.js API routes. هنگام ساخت یا بازبینی Node.js، Express یا Next.js API routes و data access آنها استفاده کنید.
25	`benchmark`	اندازه‌گیری performance baselines و تشخیص regressionها در browser Core Web Vitals (LCP، INP، CLS، page weight)، latency percentile API endpoint و build/test feedback times، با مقایسهٔ before/after در ecc/benchmarks JSON git-tracked. هنگام بررسی page speed، پاسخ به گزارش‌های 'it feels slow'، تأیید launch performance targets یا مقایسهٔ stack alternatives استفاده کنید.
26	`benchmark-methodology`	امتیازدهی یک scoped competitor set به profile cards قابل مقایسه: نه dimension وزن‌دار (positioning، voice، visual craft، offer packaging، evidence، enterprise-readiness، thought leadership، pricing، client tension) با rubricهای ۱-۵ evidence-anchored و یک tension 2x2 plot. هنگام benchmarking یا scoring competitors، ساخت competitive comparison matrix یا grading rival positioning قبل از assembling report استفاده کنید.
27	`benchmark-optimization-loop`	تبدیل درخواست‌های 'make it faster' به یک bounded measured optimization loop — ابتدا baseline، سپس generate one-hypothesis variants، benchmark هر کدام در برابر correctness gate و promote سریع‌ترین variant امن با commands قابل reproduce. هنگامی که خواسته می‌شود چیزی سریع‌تر شود، variantهای متعدد امتحان شود، recursive optimization اجرا شود، latency/throughput/cost benchmark شود یا بهترین implementation با تست‌های measured مکرر انتخاب شود استفاده کنید.
28	`bengali-nlp`	الگوهای پردازش متن Bengali (Bangla) شامل Unicode normalization، script detection، tokenization، conjunct handling و Bangla-specific NLP best practices برای AI applications.

29	`blender-motion-state-inspection`	هنگام inspect کردن characterها، rigها، poseها، facing direction، ground contact، animation retargeting یا model-vs-motion alignment در Blender که screenshotها کافی نیستند استفاده کنید.
30	`blueprint`	تبدیل یک objective یک-خطی به یک construction plan گام‌به‌گام برای پروژه‌های multi-session، multi-engineering با agent: step one-PR-sizedهای context briefها، dependency graph، self-contained با parallel-step و detection، adversarial review gate و plan mutation protocol. هنگام planning یک feature بزرگ، refactor یا roadmap که multiple PRها یا sessionها را span می‌کند استفاده کنید؛ نه برای single-PR taskها.
31	`brand-discovery`	اجرای یک brand identity interview ساختاریافته، resumable و multi-session در ۸ module (purpose، positioning، audience، personality، voice، narrative، founder tension) با استفاده از 5 Whys، laddering و projective techniques، persist کردن answerها به disk و تولید brandbook.md (90_SYNTHESIS.md). هنگام ایجاد یا repositioning یک brand، briefing designerها یا writerها یا explicit کردن implicit founder knowledge استفاده کنید.
32	`brand-voice`	ساخت یک source-derived writing style profile از postها، essayها، launch noteها، docs یا site copy واقعی، سپس reuse آن profile در content، outreach و social workflows. هنگامی که کاربر voice consistency بدون generic AI writing tropes می‌خواهد استفاده کنید.
33	`brownfield-retrospective`	اجرای ECC process engineering به‌صورت retroactive روی یک project slice موجود. audit وضعیت فعلی، reconstruct تصمیمات گذشته، تولید four-axis gap register (thinking، coding practice، missing features، technology selection)، سپس اجرای fixها forward از طریق plan-TDD-review pipeline. هنگام adopt کردن ECC mid-project، تحویل گرفتن codebase ارثی یا پرسیدن 'چه چیزی missed شد' استفاده کنید.
34	`browser-qa`	اجرای automated post-deploy UI verification با browser automation MCP (claude-in-chrome، Playwright یا Puppeteer): console-error و Core Web Vitals smoke checks، form و auth-flow interaction tests، screenshot و visual regression در سه breakpoint و axe-core accessibility audits که به verdict SHIP/DO-NOT-SHIP ختم می‌شود. هنگام تست feature deployشده روی staging/preview، قبل از shipping frontend changes، بازبینی frontend PR یا بررسی responsive layout و accessibility استفاده کنید.
35	`bun-runtime`	Bun به‌عنوان bundler، package manager، runtime و test runner. چه زمانی migration notes، Bun vs Node و Vercel support.
36	`canary-watch`	monitor و verify یک deployed URL پس از releaseها — بررسی HTTP endpoints، SSE streams، static assets، console errors و performance regressions پس از deploy، merge یا dependency upgrade. Smoke/canary/post-deploy verification.
37	`carrier-relationship-management`	مدیریت truckload، LTL و intermodal carrier portfolioها: sourcing و FMCSA vetting، freight rate و fuel-surcharge negotiation، RFPs، allocation و carrier scorecards، routing guides و renewals. هنگام onboarding carrierها، اجرای freight RFPs، مذاکرهٔ rates، ارزیابی carrier performance، reallocate کردن freight یا ساخت freight strategy استفاده کنید.
38	`change-impact`	تحلیل impact مبتنی بر evidence قبل از تغییر interfaces، data، code یا governance documents، سپس مقایسهٔ planned impact با actual diff پس از implementation. پوشش call paths، data و schema، API contracts، tests، documentation، ADRs، deployment، migration و rollback. برای cross-module یا high-risk changes، database migrations، authentication، public interfaces، blast-radius questions، pre-change checks و post-implementation alignment استفاده کنید.
39	`cisco-ios-patterns`	الگوهای review Cisco IOS و IOS-XE برای ACL placement، wildcard masks، config hierarchy، show commands، interface hygiene و safe change-window verification. هنگام خواندن، نوشتن یا بازبینی Cisco IOS/IOS-XE configuration یا planning change window استفاده کنید.
40	`ck`	Persistent per-project memory برای /ck Claude Code (Context Keeper) driven by deterministic Node.js commands: init، save، resume، info، list، forget و v1-to-v2 migrate، به‌علاوهٔ SessionStart hook که یک compact project brief inject می‌کند. هنگامی که context باید بین sessionها survive کند، ذخیرهٔ session state با next steps و decisions، resume کردن جایی که session قبلی متوقف شد یا pickup کردن project بدون re-explaining استفاده کنید.

41	`claude-devfleet`	هماهنگی multi-agent coding tasks از طریق plan — Claude DevFleet کردن projects، dispatch کردن parallel agents در isolated worktrees، monitor کردن progress و خواندن structured reports. هنگام dispatch کردن parallel coding agents در isolated worktrees و tracking reportهای آنها استفاده کنید.
42	`click-path-audit`	trace کردن هر user-facing button/touchpoint از طریق full state change sequence آن برای یافتن bugهایی که functions به‌صورت individual کار می‌کنند اما cancel each other out می‌کنند، final state اشتباه تولید می‌کنند یا UI را در inconsistent state می‌گذارند. هنگامی که systematic debugging هیچ bugی نیافت اما users broken buttons گزارش می‌دهند یا پس از major refactor touching shared state stores استفاده کنید.
43	`clickhouse-io`	الگوهای ClickHouse database، query optimization، analytics و data engineering best practices برای high-performance analytical workloads. هنگام نوشتن ClickHouse schemas یا queries یا وقتی analytical query خیلی کند است استفاده کنید.
44	`code-tour`	ایجاد فایل‌های walkthrough — `.tour` CodeTourهای step-by-step، persona-targeted با real file و line anchors. برای onboarding tours، architecture walkthroughs، PR tours، RCA tours و "explain how this works" structured requests. هنگامی که کاربر PR tour، onboarding walkthrough، code tour یا explanation نحوهٔ کار یک subsystem می‌خواهد استفاده کنید.
45	`codebase-onboarding`	تحلیل یک codebase ناآشنا و تولید structured onboarding guide با architecture map، key entry points، conventions و starter CLAUDE.md. هنگام join کردن به project جدید یا setup Claude Code برای اولین بار در یک repo استفاده کنید.
46	`codehealth-mcp`	Code Health ساختاری real-time از طریق review — CodeScene MCP قبل از edits، تأیید score deltas پس از changes، gate کردن commits و PRs. هنگام بازبینی code quality، refactoring، بررسی degradation فایل توسط AI changes یا قبل از commit/PR استفاده کنید.
47	`coding-standards`	conventions baseline cross-project برای naming، readability، immutability و code-quality review. از detailed frontend یا backend skills برای framework-specific patterns استفاده کنید. هنگام بازبینی code quality یا naming بدون framework-specific skill استفاده کنید.
48	`competitive-platform-analysis`	هنگام scoping یک competitive landscape — شناسایی، categorise کردن و score-filtering یک competitor set قبل از هر benchmarking. تصمیم می‌گیرد چه کسی competitor محسوب می‌شود، کدام tier و کدام sources mine شوند. اولین گام در three-skill competitive pipeline؛ قبل از benchmark-methodology.
49	`competitive-report-structure`	assembly کردن scored competitor profile cards (از benchmark-methodology) به یک competitive report با decision-grade landscape map، competitor tiers، benchmarking matrix، white-space analysis، strategic recommendations و team alignment trigger questions. هنگام ارائهٔ competitive findings به leadership یا board، نوشتن competitive landscape report یا به‌عنوان گام final competitive analysis pipeline استفاده کنید.
50	`compose-multiplatform-patterns`	الگوهای Compose Multiplatform و Jetpack Compose برای پروژه‌های KMP — state management، navigation، theming، performance و platform-specific UI. هنگام ساخت Compose یا Jetpack Compose UI، state یا navigation یا theming در KMP استفاده کنید.
51	`config-gc`	Garbage collection برای Claude Code configuration. به‌صورت دوره‌ای ~/.claude/ را (skills، memory، hooks،) redundant، stale، orphaned برای (permissions، MCP servers، caches) یا low-value items scan می‌کند، سپس user را از طریق confirm-each-deletion cleanup هدایت می‌کند. هنگامی که کاربر می‌گوید 'clean up my config'، 'my .claude is bloated'، 'too many skills'، 'audit my setup'، ''config GC یا periodic config review می‌خواهد استفاده کنید.
52	`configure-ecc`	اجرای ECC setup wizard مکالمه‌ای داخل harness فعلی: inventory install، collect scope (user/project/local) و hook mode (off/minimal/standard/strict) در Claude Code، استفاده از Codex native plugin lifecycle یا install project surface تحت ./.kimi-code، سپس apply و preview، verify. هنگام install، update، reconfigure یا repair ECC installation، تغییر hook profiles یا move ECC بین install scopes استفاده کنید.
53	`connections-optimizer`	reorganize کردن network X و LinkedIn کاربر با add/follow recommendations، review-first pruning و channel-specific warm outreach drafted در voice واقعی کاربر. هنگامی که کاربر می‌خواهد followingها را clean کند، به سمت priorities فعلی grow کند یا social graph را حول higher-signal relationships rebalance کند استفاده کنید.

54	`content-engine`	ایجاد content systems platform-native برای X، LinkedIn، TikTok، YouTube، newsletters و repurposed multi-platform campaigns. هنگامی که کاربر social posts، threads، scripts، content calendars یا یک source asset می‌خواهد adapted cleanly across platforms استفاده کنید.
55	`content-hash-cache-pattern`	cache کردن نتایج expensive file processing با استفاده از SHA-256 content hashes — path-independent، auto-invalidating، با service layer separation. هنگامی که repeated file processing کند است و results باید cached و invalidated by content (نه path) شوند استفاده کنید.
56	`context-and-decisions`	Govern کردن stable domain context با یک CONTEXT.md narrowly scoped وقتی needed، و route کردن hard-to-reverse decisions به skill architecture-decision-records موجود ECC. هنگامی که domain terms نیاز به repeated explanation دارند، یک term معانی multiple دارد، unclear language باعث module-boundary mistakes می‌شود یا domain semantics باید separate از architecture decisions بمانند استفاده کنید.
57	`context-budget`	route کردن live context checks به native Codex یا Claude Code meter بدون scan کردن files، و انجام یک bounded static component audit فقط وقتی explicitly requested. برای context-budget requests دربارهٔ remaining capacity یا installed context overhead استفاده کنید.
58	`continuous-agent-loop`	الگوهای continuous autonomous agent loops با quality gates، evals و recovery controls. هنگام اجرای agent loop که باید self-check کند، gate on evals و recover from failures استفاده کنید.
59	`continuous-learning`	[DEPRECATED - از continuous-learning-v2 استفاده کنید] Legacy v1 stop-hook skill extractor. v2 strict superset با instinct-based، project-scoped، hook-reliable learning است. v1 را invoke نکنید.
60	`continuous-learning-v2`	سیستم یادگیری instinct-based که sessions را از طریق hooks observe می‌کند، atomic instincts با confidence scoring ایجاد می‌کند و آنها را به skills/commands/agents evolve می‌کند. v2.1 project-scoped instincts برای جلوگیری از cross-project contamination اضافه می‌کند. هنگام capture کردن lessons از session، manage کردن instincts یا promote کردن آنها به skills، commands یا agents استفاده کنید.
61	`contract-first`	هماهنگی frontend/backend یا service-to-service work از طریق یک authoritative machine-checkable contract (OpenAPI، AsyncAPI، Protocol Buffers یا JSON Schema)، با generated consumer types و contract-verified integration. هنگامی که parallel consumer و provider work باید یک API یا event schema را evolve کنند بدون field drift، mock/production shape mismatch یا redefining silent interface استفاده کنید.
62	`conventional-commit-review`	بازبینی staged changes و draft یک Conventional Commits-compliant commit message با scope، type و length درست. قبل از commit یا هنگامی که کاربر می‌خواهد commit message بنویسد، fix یا review کند استفاده کنید.
63	`cost-aware-llm-pipeline`	الگوهای بهینه‌سازی cost برای LLM API usage — model routing بر اساس task complexity، budget tracking، retry logic و prompt caching. هنگامی که LLM spend باید کم شود یا routing tasks بین model tiers و budgets استفاده کنید.
64	`cost-tracking`	track و report Claude Code token usage، spending و budgets از local ECC cost-tracker metrics log. هنگامی که کاربر دربارهٔ budgets، tokens، usage، spending، costs یا cost breakdowns by model، session یا date می‌پرسد استفاده کنید.
65	`council`	برگزاری یک council چهار-صدا برای tradeoffs ambiguous، decisions و go/no-go calls. هنگامی که multiple valid paths وجود دارد و نیاز به structured disagreement قبل از choosing استفاده کنید.
66	`council-multi-model`	افزودن یک external Codex critique اختیاری پس از اینکه council موجود یک decision draft تولید کرده. هنگامی که یک decision ambiguous، high-consequence از یک separate model invocation برای attempt کردن break synthesis بهره می‌برد استفاده کنید. نیاز به explicit consent قبل از ارسال compact draft و disagreement به OpenAI دارد.
67	`counterparty-channel-discipline`	Per-channel strict prompts، mention gating، silent observation و communication autonomy policy برای agentهایی که در shared channels با external counterparties قرار دارند. هنگامی که agent به group chats، shared channels یا DMs می‌پیوندد که outsiders می‌توانند هر message را بخوانند و باید only when addressed صحبت کند، هرگز internal context leak نکند و risky content را به draft-only approval route کند استفاده کنید.

68	`cpp-coding-standards`	استانداردهای کدنویسی C++ بر اساس C++ Core Guidelines (isocpp.github.io). هنگام نوشتن، بازبینی یا refactoring C++ code برای enforce کردن safe، modern و idiomatic practices استفاده کنید.
69	`cpp-testing`	فقط هنگام نوشتن/updating/fix کردن C++ tests، configure کردن GoogleTest/CTest، تشخیص failing یا flaky tests یا افزودن coverage/sanitizers استفاده کنید.
70	`crosspost`	توزیع content multi-platform در X، LinkedIn، Threads و Bluesky. content را با استفاده از per platform content-engine patterns adapt می‌کند. هرگز identical content cross-platform post نمی‌کند. هنگامی که کاربر می‌خواهد content را در social platforms توزیع کند استفاده کنید.
71	`csharp-testing`	الگوهای تست C# و .NET با xUnit، FluentAssertions، mocking، integration tests و test organization best practices. هنگام نوشتن یا بازبینی xUnit tests، mocks یا integration tests در C#/.NET استفاده کنید.
72	`customer-billing-ops`	عملیات customer billing workflows مانند subscriptions، refunds، churn triage، billing-portal recovery و plan analysis با استفاده از connected billing tools مانند Stripe. هنگامی که کاربر نیاز به help کردن customer، inspect کردن subscription state یا manage کردن revenue-impacting billing operations دارد استفاده کنید.
73	`customs-trade-compliance`	expertise codified customs و HS/HTS tariff classification — trade compliance با GRI rules، commercial invoices و entry documentation، Incoterms 2020، FTA qualification و duty optimization (USMCA، RCEP، FTZs، drawback)، denied-party screening و penalty mitigation در US، EU، UK و APAC. هنگام classify کردن goods، آماده‌سازی import/export documentation، screening restricted parties، پاسخ به customs audits یا CF-28/penalty notices یا optimizing duties استفاده کنید.
74	`dart-flutter-patterns`	الگوهای production-ready Dart و Flutter پوشش‌دهندهٔ null safety، immutable state با Freezed، async composition، widget architecture، state management (BLoC، Riverpod، Provider)، GoRouter navigation، auth guards، Dio networking، error handling و testing. هنگام نوشتن یا بازبینی Dart و Flutter code — state، widgets، navigation، networking یا architecture استفاده کنید.
75	`dashboard-builder`	ساخت monitoring dashboards که به سؤالات واقعی operator پاسخ می‌دهند برای Grafana، SigNoz و platforms مشابه. هنگام تبدیل metrics به یک dashboard کاربردی (نه vanity board) استفاده کنید.
76	`data-scraper-agent`	ساخت یک AI-powered data collection agent کاملاً automated برای هر job boards، prices، public source — news، GitHub، sports و هر چیز دیگر. روی schedule اجرا می‌شود، data را با free LLM (Gemini Flash) enrich می‌کند، results را در Notion/Sheets/Supabase ذخیره می‌کند و از user feedback یاد می‌گیرد. ۱۰۰٪ free روی GitHub Actions. هنگامی که کاربر می‌خواهد public data را automatically monitor، collect یا track کند استفاده کنید.
77	`data-throughput-accelerator`	تشخیص و تسریع large data movement — ingestion، backfill، export، ETL، warehouse loading، manifest catch-up و table synchronization — با isolate کردن benchmark، true bottleneck کردن variants و codify کردن fastest path با یک hard accounting block که rows و timestamps را cohere ثابت می‌کند. هنگامی که یک pipeline یا backfill خیلی کند است و باید بدون از دست دادن data correctness سریع‌تر شود استفاده کنید.
78	`database-migrations`	الگوهای migration امن و zero-downtime reversible: forward-only production changes، expand-contract renames، concurrent indexes، batched backfills و per-tool workflows برای PostgreSQL، Prisma، Drizzle، Kysely، Django و golang-migrate. هنگام نوشتن schema یا data migration، افزودن column یا index به table بزرگ، rollback planning یا آماده‌سازی zero-downtime deploy استفاده کنید.
79	`deep-research`	تولید cited research reports از multiple web sources با استفاده از firecrawl و exa MCP tools — plan sub-questions، search و deep-read sources، سپس synthesize کردن findings با inline citations و confidence levels. هنگامی که کاربر دربارهٔ یک topic in depth research می‌خواهد، یا deep dive، investigation، competitive analysis، technology evaluation، market sizing یا due diligence روی یک company استفاده کنید.
80	`deepseek-harness-setup`	install و operate ECC روی DeepSeek Harness (DSH): native skill roots (~/.dsh/skills، .agents/skills)، bridge @deepseek-ai/dsh-hooks-claude-code برای command hooks، bare-insert patch mounting، generator usage، event-support limits و update workflow. هنگام setup ECC روی DeepSeek Harness، wiring ECC hooks به DSH profile یا debugging چرا ECC skills یا hooks در DSH session ظاهر نمی‌شوند استفاده کنید.

81	`defi-amm-security`	checklist امنیتی برای Solidity AMM contracts، liquidity pools و swap flows. پوشش reentrancy، CEI ordering، donation/inflation attacks، oracle manipulation، slippage، admin controls و integer math. هنگام audit یا نوشتن Solidity AMM، liquidity pool یا swap code استفاده کنید.
82	`delivery-gate`	Stop hook که Claude را از finish کردن تا زمانی که quality checks pass می‌شوند block می‌کند. rationalization patterns (surface text heuristics)، stale learning logs (filesystem mtime) و low disk space را تشخیص می‌دهد. self-audit را با mechanically enforcing learning capture habits complement می‌کند. هنگامی که Claude باید mechanically blocked شود از declare کردن work finished قبل از اینکه quality checks و learning capture واقعاً pass شوند استفاده کنید.
83	`deployment-patterns`	Deployment workflows، CI/CD pipeline patterns، Docker containerization، health checks، rollback strategies و production readiness checklists برای web applications. هنگام containerizing، setup CI/CD یک app یا بررسی production readiness قبل از release استفاده کنید.
84	`design-system`	تولید یک design system از codebase موجود یا audit یکی برای visual consistency: extract tokens (colors، typography، spacing، shadows) به design-tokens.json و CSS custom properties با DESIGN.md rationale و interactive HTML preview، score کردن UI در ۱۰ dimension و flag کردن AI-slop patterns. هنگام شروع design system، audit visual consistency قبل از redesign یا بازبینی PR که styling را touch می‌کند استفاده کنید.
85	`dev-team`	شبیه‌سازی یک dev team session collaborative که multiple role-based personas (PM، Architect، Developer، QA) به یک problem در یک session پاسخ می‌دهند. هنگام طراحی feature، بازبینی proposal یا onboarding initiative جدید و نیاز به multi-role perspective بدون switching agents manually استفاده کنید.
86	`django-celery`	الگوهای async task Django + Celery — configuration، task design، beat scheduling، retries، canvas workflows، monitoring و testing. هنگام افزودن background jobs، scheduled tasks یا async processing به Django app استفاده کنید.
87	`django-patterns`	الگوهای معماری Django، REST API design با DRF، ORM best practices، caching، signals، middleware و production-grade Django apps. هنگام ساخت یا بازبینی Django apps، DRF APIs، ORM queries یا caching استفاده کنید.
88	`django-security`	Django security best practices، authentication، authorization، CSRF protection، SQL injection prevention، XSS prevention و secure deployment configurations. هنگام بازبینی Django authentication، authorization، input handling یا deployment settings استفاده کنید.
89	`django-tdd`	استراتژی‌های تست Django با pytest-django، TDD methodology، factory_boy، mocking، coverage و testing Django REST Framework APIs. هنگام نوشتن Django یا DRF tests با pytest-django یا driving یک Django feature test-first استفاده کنید.
90	`django-verification`	اجرای full Django verification loop — environment check، mypy/ruff/black linting، migration safety، pytest با coverage targets، pip-audit، settings و bandit security scans و logging review و diff review — تولید phased pass/fail report قبل از release یا PR. هنگام آماده‌سازی Django pull request، validation migrations یا coverage running pre-deploy readiness checks استفاده کنید.
91	`dmux-workflows`	Multi-agent orchestration با استفاده از dmux (tmux pane manager برای AI agents). الگوهای parallel agent workflows در Claude Code، Codex، OpenCode و سایر harnesses. هنگام اجرای multiple agent sessions به‌صورت parallel یا هماهنگی multi-agent development workflows استفاده کنید.
92	`docker-patterns`	الگوهای Docker و Docker Compose برای container local development، hardened CLI installer harnesses، networking، volumes، security و multi-service orchestration. هنگام ایجاد یا بازبینی Dockerfiles و Compose services، testing installers در Linux distributions یا validation native macOS/Windows planning استفاده کنید.
93	`docs-governance`	route کردن broad documentation-governance requests به existing ECC skills و اجرای read-only audit، opt-in، ADR indexes، links، mapped documentation roles و evidence references. هنگامی که user documentation، documentation integrity، project-knowledge organization، governance یا deterministic docs audit بدون نام بردن skill narrower می‌خواهد استفاده کنید.

94	`documentation-lookup`	استفاده از up-to-date library و framework docs از طریق Context7 MCP به جای training data. برای setup questions، API references، code examples یا هنگامی که user نام یک framework (React، Next.js، Prisma) را می‌برد activate می‌شود.
95	`dotnet-patterns`	الگوهای ایدیوماتیک C# و .NET، conventions، dependency injection، async/await و best practices برای ساخت robust، maintainable .NET applications. هنگام نوشتن یا بازبینی C#/.NET code — DI، async general یا conventions استفاده کنید.
96	`dynamic-workflow-mode`	طراحی task-local harnesses، eval gates و reusable skill extraction برای Claude dynamic workflow mode و سایر adaptive agent harnesses. هنگام ساخت task-local harness، افزودن eval gates یا extract کردن reusable skill از ad-hoc work استفاده کنید.
97	`e2e-testing`	الگوهای Playwright E2E testing، Page Object Model، configuration، CI/CD integration، artifact management و flaky test strategies. هنگام نوشتن Playwright tests، structure کردن page objects یا fix کردن flaky E2E runs در CI استفاده کنید.
98	`ecc-guide`	پاسخ به سؤالات دربارهٔ ECC با خواندن live repo surface — agents، skills، commands، hooks، rules، install profiles و docs — به جای memory. هنگامی که user می‌پرسد ECC چه چیزی شامل می‌شود، چگونه install یا reset کند، کدام skill یا command fit یک task است یا چگونه project onboarding کار می‌کند استفاده کنید.
99	`ecc-recipes`	map کردن یک workflow توصیف‌شده به right ECC command group با run-order و stop condition، یا browse کردن همهٔ live command-group recipe families از commands directory. فقط advisory — هرگز execute نمی‌کند. هنگامی که پرسیده می‌شود کدام commands یک workflow را run می‌کنند، sequence command برای task یا list کردن ECC pipelines استفاده کنید.
100	`ecc-tools-cost-audit`	گردش‌کار audit burn و billing ECC Tools مبتنی بر evidence. هنگام investigation runaway PR creation، quota bypass، premium-model leakage، duplicate jobs یا GitHub App cost spikes در repo ECC Tools استفاده کنید.
101	`email-ops`	گردش‌کار send verification، drafting، triage و sent-mail-safe follow-up mailbox مبتنی بر evidence برای ECC. هنگامی که کاربر می‌خواهد email را organize کند، draft یا send از طریق real mail surface یا prove کند چه چیزی در Sent landed استفاده کنید.
102	`energy-procurement`	procure کردن electricity و natural gas برای facilities commercial و industrial: tariff و rate-schedule optimization، demand-charge mitigation، supplier RFPs، fixed/index/block-and-index hedging، renewable PPA و REC evaluation و sustainability reporting. هنگام procure کردن energy، optimizing utility tariffs، managing demand charges، evaluating PPAs یا ساخت energy budgets و hedge strategies استفاده کنید.
103	`enterprise-agent-ops`	cloud-hosted — runtime lifecycle (start، pause، stop، یا agent systems long-lived برای controls operational restart)، observability (logs، metrics، traces)، least-privilege safety scopes و kill switches، و rollout/rollback با change management و audit logs و success/cost metrics. هنگام اجرای production agent fleets روی PM2، systemd یا containers که نیاز به monitoring، incident response یا deployment gates دارند استفاده کنید.
104	`error-handling`	الگوهای robust error handling در TypeScript، Python و Go. پوشش typed errors، error boundaries، retries، circuit breakers و user-facing error messages. هنگام طراحی error types، retries، circuit breakers یا user-facing failure messages در TypeScript، Python یا Go استفاده کنید.
105	`esign-field-placement`	method deterministic برای قرار دادن signature، date و text fields در یک web e-signature composer از طریق browser automation session، با استفاده از یک signature page ثابت، numeric Location panel coordinates به جای drag و save-as-draft default. هنگام automate کردن envelope preparation برای generated agreements و نیاز به repeatable field positions، correct per-recipient ownership و hard gate قبل از send یا sign استفاده کنید.
106	`eval-harness`	framework eval-driven development (EDD) برای AI coding sessions — تعریف capability و regression evals قبل از coding، grade کردن با code-based، model-based، rule یا human graders و track کردن pass@k و pass^k reliability. هنگام تعریف pass/fail criteria برای agent tasks، اندازه‌گیری agent reliability، ساخت regression suites برای prompt یا agent changes یا benchmarking بین model versions استفاده کنید.
107	`evm-token-decimals`	جلوگیری از silent decimal mismatch bugs در EVM chains. پوشش runtime decimal lookup، chain-aware bridged-token precision drift، caching و safe normalization برای bots، dashboards و DeFi tools. هنگام handle

		کردن token amounts در EVM chains یا وقتی یک balance، price یا transfer amount چند order of magnitude off است استفاده کنید.
108	`exa-search`	Neural search از طریق Exa MCP برای web، code و company research. هنگامی که کاربر web search، code examples، company intel، people lookup یا AI-powered deep research می‌خواهد Exa neural search engine با استفاده کنید.
109	`fal-ai-media`	تولید media یکپارچه از طریق fal.ai MCP — image، video و audio. پوشش text-to-image (Nano Banana)، text/image-to-video (Seedance، Kling، Veo 3)، text-to-speech (CSM-1B) و video-to-audio (ThinkSound). هنگامی که کاربر می‌خواهد images، videos یا audio با AI تولید کند استفاده کنید.
110	`fastapi-patterns`	پوشش‌دهندهٔ best practices FastAPI project structure، Pydantic v2 schemas، dependency injection، async handlers، authentication، authorization، transactional service layers و testing با httpx و pytest. هنگام ساخت یا بازبینی FastAPI apps — Pydantic schemas، dependencies، async handlers، auth یا tests استفاده کنید.
111	`finance-billing-ops`	گردش‌کار team-billing، refunds، pricing، revenue و billing-model truth مبتنی بر evidence برای ECC. هنگامی که کاربر duplicate-charge diagnosis، pricing comparison، sales snapshot یا code-backed billing reality (نه generic payments advice) می‌خواهد استفاده کنید.
112	`flox-environments`	ایجاد reproducible، cross-platform (macOS/Linux) development environments با Flox، یک declarative Nix-based environment manager. هنگام setup project toolchains، install system-level dependencies (compilers، databases، native libs)، pin کردن exact package versions برای local team، onboarding developers، running services (PostgreSQL، Redis، Kafka) یا حل 'works on my machine' استفاده کنید.
113	`flutter-dart-code-review`	پوشش‌دهندهٔ checklist code review library-agnostic Flutter/Dart widget best practices، state management patterns (BLoC، Riverpod، Provider، GetX، MobX، Signals)، Dart idioms، performance، accessibility، security و clean architecture. هنگام بازبینی Flutter یا Dart code، هر state management library که پروژه استفاده می‌کند استفاده کنید.
114	`foundation-models-on-device`	Apple FoundationModels framework برای on-device LLM — text generation، guided generation با @Generable، tool calling و snapshot streaming در iOS 26+. هنگام افزودن on-device LLM features با Apple FoundationModels روی iOS 26+ استفاده کنید.
115	`frontend-a11y`	الگوهای accessibility برای React و Next.js — semantic HTML، ARIA attributes، form labeling، keyboard navigation، focus management و screen reader support. هنگام ساخت یا بازبینی forms، modals، dropdowns، tooltips یا fix، tabs کردن a11y lint یا code-review findings با wiring keyboard navigation و focus management استفاده کنید.
116	`frontend-design-direction`	تنظیم یک ECC-specific frontend design direction برای production UI work. هنگام ساخت یا بهبود websites، dashboards، applications، components، landing pages، visual tools یا هر web UI که نیاز به stronger product-specific design judgment دارد استفاده کنید.
117	`frontend-patterns`	الگوهای توسعهٔ frontend برای state management، performance optimization React، Next.js و UI best practices. هنگام ساخت یا بازبینی Next.js components، state یا React یا render performance استفاده کنید.
118	`frontend-slides`	ایجاد stunning، animation-rich HTML presentations از scratch یا تبدیل PowerPoint files. هنگامی که کاربر می‌خواهد presentation بسازد، PPT/PPTX را به web تبدیل کند یا slides برای talk/pitch بسازد استفاده کنید.
119	`fsharp-testing`	الگوهای تست F# با xUnit، FsUnit، Unquote، FsCheck property-based testing، integration tests و test organization best practices. هنگام نوشتن F# tests با xUnit، FsUnit، Unquote یا FsCheck استفاده کنید.
120	`gan-style-harness`	GAN-inspired Generator-Evaluator agent harness برای ساخت high-quality applications به‌صورت autonomous. بر اساس Anthropic's March 2026 harness design paper. هنگامی که یک feature باید به‌صورت autonomous از طریق generator و evaluator iteration تا زمانی که quality bar را pass کند ساخته شود استفاده کنید.
121	`gateguard`	PreToolUse fact-forcing gate که اولین Edit/Write/Bash (شامل MultiEdit و NotebookEdit) attempt را تا زمانی که agent concrete facts (importers، data schemas، verbatim user instruction) ارائه دهد deny می‌کند، سپس retry

		را allow می‌کند. A/B-tested at +2.25 quality points. هنگام enabling یا configuring GateGuard hook، exempt کردن paths از طریق env vars یا handling first-touch denials استفاده کنید.
122	`generating-python-installer`	Windows: Nuitka extreme compilation، dist slimming، DLL برای expert Python installer commercial-grade footprint analysis و Inno Setup packaging برای ship کردن smallest، fastest installers. هنگامی که Python app باید به‌عنوان minimal، fast-starting Windows installer ship شود (نه basic script-to-exe conversion) استفاده کنید.
123	`git-workflow`	الگوهای Git workflow شامل branching strategies، commit conventions، clean و readable نگه داشتن history، tidying local commits قبل از merging، merge vs rebase، conflict resolution و collaborative development best practices. هنگام انتخاب branching strategy، نوشتن commit conventions، cleanup history قبل از PR، تصمیم merge vs rebase یا resolve کردن conflicts استفاده کنید.
124	`github-ops`	عملیات GitHub repository، automation و management. Issue triage، PR management، CI/CD operations، release management و security monitoring با استفاده از gh CLI. هنگامی که کاربر می‌خواهد GitHub issues، PRs، CI status، releases، contributors، stale items یا هر GitHub operational task فراتر از simple git commands را manage کند استفاده کنید.
125	`golang-patterns`	الگوها و conventions ایدیوماتیک Go، best practices برای ساخت robust، efficient و maintainable Go applications. هنگام نوشتن یا بازبینی Go code و idiomatic structure یا conventions در question استفاده کنید.
126	`golang-testing`	الگوهای تست Go شامل table-driven tests، subtests، benchmarks، fuzzing و test coverage. TDD methodology با idiomatic Go practices. هنگام نوشتن table-driven cases، subtests، benchmarks، fuzzing یا Go tests — coverage استفاده کنید.
127	`google-workspace-ops`	عملیات در Google Drive، Docs، Sheets و Slides به‌عنوان یک workflow surface برای decks، trackers، plans و shared documents. هنگامی که کاربر نیاز به find، summarize، edit، migrate یا cleanup Google Workspace assets بدون dropping به raw tool calls دارد استفاده کنید.
128	`growth-log`	نوشتن growth log entries که reusable patterns را از completed work extract می‌کنند — root cause، transferable rule و recognizable signal — به جای diary-style event narration، با template ۴-۸ جمله و merge-duplicates discipline. هنگام capture کردن what was learned پس از debugging session، failure یا complex task، rollback، review کردن progress در یک period یا وقتی delivery gate می‌پرسد what was learned استفاده کنید.
129	`healthcare-cdss-patterns`	الگوهای توسعهٔ Clinical Decision Support System (CDSS). Drug interaction checking، dose validation، clinical scoring (NEWS2، qSOFA)، alert severity classification و integration به EMR workflows. هنگام ساخت clinical decision support — drug interaction checks، dose validation، clinical scoring یا alert severity استفاده کنید.
130	`healthcare-emr-patterns`	الگوهای توسعهٔ EMR/EHR برای healthcare applications. Clinical safety، encounter workflows، prescription generation، clinical decision support integration و accessibility-first UI برای medical data entry. هنگام ساخت EMR/EHR features مانند encounter workflows، prescription generation یا clinical data entry UI استفاده کنید.
131	`healthcare-eval-harness`	Patient safety evaluation harness برای healthcare application deployments. Automated test suites برای CDSS accuracy، PHI exposure، clinical workflow integrity و integration compliance. Deployments را روی safety failures block می‌کند. هنگامی که یک healthcare deployment باید gated شود روی patient-safety tests برای CDSS accuracy، PHI exposure و workflow integrity استفاده کنید.
132	`healthcare-phi-compliance`	الگوهای PHI و PII compliance برای healthcare applications: data classification، row-level access control، tamper-proof audit trails، schema tagging و common leak vectors مانند URLs، logs و browser storage. هنگامی که code patient یا clinician data را touch می‌کند، هنگام implement کردن HIPAA/GDPR access controls یا audit کردن healthcare system برای data exposure استفاده کنید.
133	`hermes-imports`	تبدیل local Hermes operator workflows به sanitized ECC skills و release-pack artifacts. هنگام آماده‌سازی یک Hermes workflow برای public ECC reuse بدون leak کردن credentials، private workspace state یا local-only paths استفاده کنید.

134	`hexagonal-architecture`	طراحی، پیاده‌سازی و refactor سیستم‌های Ports & Adapters با clear domain boundaries، dependency inversion و testable use-case orchestration در TypeScript، Java، Kotlin و Go services. هنگام introducing یا refactoring به سمت Ports and Adapters یا وقتی domain logic با I/O entangle شده است استفاده کنید.
135	`hipaa-compliance`	Entrypoint HIPAA-specific برای healthcare privacy و security work. هنگامی که task explicitly حول HIPAA، PHI یا handling، covered entities، BAAs، breach posture یا US healthcare compliance requirements framed شده استفاده کنید.
136	`homelab-network-readiness`	Readiness checklist برای homelab VLAN segmentation، local DNS filtering (Pi-hole، AdGuard Home) و WireGuard-style remote access. هنگام planning یا review کردن splitting flat home network changes — management VLANs، move یا trusted، IoT، guest به network به DHCP کردن یا local resolver به افزودن VPN access — قبل از تغییر router، firewall، DHCP یا VPN configuration استفاده کنید.
137	`homelab-network-setup`	Planning عملی home و homelab network برای gateways، switches، access points، IP ranges، DHCP، reservations، DNS، cabling و common beginner mistakes. هنگام planning یا fix کردن home یا homelab network — gateway، switch، AP، IP ranges، DHCP، DNS یا cabling استفاده کنید.
138	`homelab-pihole-dns`	Pi-hole installation، blocklist management، DNS-over-HTTPS setup، DHCP integration، local DNS records و troubleshooting broken DNS resolution روی home network. هنگامی که Pi-hole را task explicitly involve می‌کند — install کردن آن، manage کردن blocklists، configure کردن DoH یا DHCP، افزودن local DNS records یا diagnosing DNS resolution با Pi-hole در path استفاده کنید.
139	`homelab-vlan-segmentation`	Segmenting home networks به VLANs برای IoT، guest، trusted و server traffic با استفاده از UniFi، pfSense/OPNsense و MikroTik — شامل firewall rules، switch trunk config و wireless SSID mapping. هنگام split کردن home network به IoT، guest، trusted و server VLANs روی UniFi، pfSense/OPNsense یا MikroTik استفاده کنید.
140	`homelab-wireguard-vpn`	WireGuard VPN server setup، peer configuration، key generation، split tunneling vs full tunnel routing و remote access به home network از mobile و laptop clients. هنگام setup کردن WireGuard برای remote access به home network یا تصمیم بین split و full tunnel routing استفاده کنید.
141	`hookify-rules`	ایجاد و پیکربندی hookify rules — فایل‌های markdown با YAML frontmatter که bash، file، prompt یا stop events را با regex یا conditions match می‌کنند و warn/block messages به agent نشان می‌دهند. هنگام ایجاد یک hookify rule، نوشتن hook rule syntax، configure کردن hookify یا افزودن pattern guardrails مانند block کردن dangerous .env edits، commands یا debug code استفاده کنید.
142	`i18n-sync`	translate و synchronize کردن application JSON locale files با استفاده از source-key usage، project terminology و focused validation. هنگام افزودن keys یا source copy یا languages، updating یا reviewing missing و stale translations استفاده کنید.
143	`ig-audit`	Post-mortem روی آنچه کاربر قبلاً در Instagram post کرده — کدام reels واقعاً کار کردند، چرا و چه چیزی باید stop شود. هنگامی که کاربر Instagram insights یا past posts خود را paste می‌کند و می‌پرسد 'why did' 'what's working'، 'audit my content'، 'read my analytics'، 'this flop' یا می‌خواهد بداند چه چیزی بیشتر انجام دهد استفاده کنید.
144	`ig-caption`	نوشتن Instagram caption — خطی که از '... more' cut جان سالم به در می‌برد، search terms، single ask، body و three hashtags — و lint کردن قبل از ارسال. هنگامی که کاربر می‌گوید 'what do' 'write the caption'، 'caption this'، 'I put in the description'، یک reel یا carousel آماده دارد و نیاز به text دارد یا دربارهٔ hashtags می‌پرسد استفاده کنید.
145	`ig-carousel`	ساخت یک Instagram carousel — cover که swipe را earn می‌کند، slide-by-slide copy و فایل‌های 1080x1350 برای upload. هنگامی که کاربر می‌گوید 'turn this into a carousel'، 'swipe post'، 'slides'، 'carousel' یا ایدهٔ list-shaped یا step-shaped دارد که به‌عنوان single image می‌میرد استفاده کنید.
146	`ig-comment`	نوشتن comment روی postها و reelهای دیگران در Instagram که به‌عنوان یک person با opinion خوانده می‌شود، نه یک bot. هنگامی که کاربر post یا reel را paste می‌کند و comment می‌خواهد، می‌گوید 'engage'، 'comment on this' 'what do I say here'، 'with this' یا batch برای daily engagement round می‌خواهد استفاده کنید.
147	`ig-dm`	نوشتن Instagram DMهایی که reply می‌گیرند — keyword delivery، first message به کسی که engaged کرده، collab pitch و follow-up. هنگامی که کاربر می‌گوید 'outreach' 'what do I send them'، 'DM this person'

		'message'، 'how do I follow up'، 'pitch this brand' یا به someone specific reach out می‌کند استفاده کنید.
148	`ig-human`	strip کردن machine fingerprint از هر em dashes، AI slop words، invisible watermark characters — draft — و score کردن در برابر five-check detection panel قبل از ارسال. هر زمان text باید human به نظر برسد، هنگامی که کاربر می‌گوید 'humanize، 'does this sound like AI'، 'remove the em dashes'، 'de-slop this'، 'this sounds like ChatGPT' یا قبل از نمایش هر caption، script، comment، reply یا DM به کاربر استفاده کنید.
149	`ig-plan`	ساخت week روی Instagram — چه چیزی post کنید، کدام format، کی و با چه کسی engage کنید. هنگامی که کاربر می‌گوید 'I have nothing to post about'، 'content calendar'، 'what should I post'، 'plan my week' یا posting schedule و engagement list می‌خواهد استفاده کنید.
150	`ig-profile`	امتیازدهی Instagram profile از ۱۰۰ در برابر ۱۲-part rubric و rewrite کردن parts که points از دست می‌دهند — name field، bio، link، highlights، pinned three، grid. هنگامی که کاربر می‌گوید 'fix my optimize my profile'، 'why don't people follow me'، 'score my Instagram'، 'bio' یا profile خود را paste می‌کند و می‌پرسد چگونه خوانده می‌شود استفاده کنید.
151	`ig-reel`	نوشتن یک Instagram Reel از یک hook options — raw idea از ۲۶ formula، spoken script، on-screen text و timed beat sheet — در voice خود کاربر و scored قبل از shoot. هر زمان کاربر Reel، short-form video script، 'hook، voiceover، 'make a reel about X'، 'what should I say in this video می‌خواهد یا قصد record کردن دارد و first line را ندارد استفاده کنید.
152	`ig-reply`	handle کردن commentهای زیر reelها و postهای خود کاربر — draft replies به آنهایی که worth answering هستند، sorted by which ones are. هنگامی که کاربر commentهای خود را paste می‌کند، می‌گوید 'handle' 'reply to these'، 'my comments'، 'someone said X on my reel' یا با یک critic، hater یا lead در comments deal می‌کند استفاده کنید.
153	`ig-repurpose`	تبدیل یک long asset — YouTube video، podcast، livestream، newsletter، blog post یا client call — به یک week of reels و carousels. هنگامی که کاربر می‌گوید 'I have a repurpose this'، 'turn this into reels'، 'cut this up' 'video/podcast/transcript' یا چیزی طولانی paste می‌کند و می‌خواهد روی Instagram باشد استفاده کنید.
154	`ig-story`	نوشتن frame-by-frame sequence — day's Instagram Stories، کدام stickers کجا استفاده شوند و کدام یک people را به DMها move می‌کند. هنگامی که کاربر می‌گوید 'story sequence'، 'what do I post on my story'، 'I have nothing to story about poll idea'' یا می‌خواهد بدون post کردن چیزی sell کند استفاده کنید.
155	`ig-viral`	پیدا کردن reelهایی که right now در niche کاربر واقعاً کار می‌کنند، rank کردن آنها بر اساس اینکه هر کدام چقدر account خود را beat کرده‌اند، نام‌گذاری hook formula که هر کدام استفاده کرده و تبدیل آن به یک swipe file برای shoot. هنگامی که کاربر می‌گوید 'what are people posting in my find viral videos'، 'what's working right now'، 'why is this reel doing numbers'، 'reverse engineer this account'، 'build me a swipe file'، 'niche' یا می‌پرسد next چه بسازد و evidence ندارد استفاده کنید.
156	`inherit-legacy-style`	جلوگیری از AI style drift روی legacy projects با scanning codebase برای implicit conventions، resolving conflicts با operator یکی یکی و نوشتن یک ai-style-rules.md enforceable (Golden Files، naming rules، DONTs). به‌علاوهٔ optional CLAUDE.md hook. هنگام onboarding یک AI agent روی hand-written legacy codebase یا extracting unwritten coding rules پروژه استفاده کنید.
157	`intent-driven-development`	تبدیل ambiguous یا high-impact product و engineering changes به scoped، verifiable acceptance criteria قبل یا در کنار implementation. هنگامی که user می‌خواهد یک feature را clarify کند، acceptance criteria تعریف کند، یک security/data/migration/integration change را de-risk کند، implementation requirements برای agent دیگری آماده کند یا یک request پیچیده را testable کند استفاده کنید. برای trivial edits، straightforward fixes، active debugging، code review یا implementation requests که acceptance conditions قبلاً clear هستند trigger نکنید.
158	`inventory-demand-planning`	multi-location retailers: demand forecasting method selection، برای expertise codified demand planning، ABC/XYZ segmentation، safety stock و reorder-point optimization، promotional lift و post-promo dip estimation، seasonal transition و markdown timing. هنگام forecasting demand، setting safety stock، planning replenishment، managing promotions یا optimizing inventory levels استفاده کنید.

159	`investor-materials`	ایجاد و به‌روزرسانی pitch decks، one-pagers، investor memos، accelerator applications، financial models و fundraising materials. هنگامی که user نیاز به investor-facing documents، projections، use-of-funds tables، milestone plans یا materials که باید internally consistent در multiple fundraising assets بمانند دارد استفاده کنید.
160	`investor-outreach`	Draft کردن cold emails، warm intro blurbs، follow-ups، update emails و investor communications برای fundraising. هنگامی که user outreach به angels، VCs، strategic investors یا accelerators می‌خواهد و نیاز به concise، personalized، investor-facing messaging دارد استفاده کنید.
161	`ios-icon-gen`	تولید iOS app icons به‌عنوان PNG imagesets برای Xcode asset catalogs از SF Symbols (5000+ Apple-native) یا Iconify API (275k+ open source icons از 200+ collections). هنگام generating icons، creating icon assets، searching for icons یا adding icons to asset catalog برای iOS projects استفاده کنید.
162	`iterative-retrieval`	الگوی progressively refining context retrieval برای حل subagent context problem. هنگامی که یک subagent context مورد نیاز را ندارد و retrieval باید across passes refined شود استفاده کنید.
163	`ito-baskets`	live basket catalog، compare کردن prediction-market data. Index و skill read-only Itô basket کردن یک basket با user-supplied research یا watchlist، build کردن source-grounded market brief یا draft کردن non-executable planning worksheet. هنگامی که user می‌خواهد Itô baskets را browse یا index کند، یک basket را با notes یا thesis compare کند، prediction-market events/venues/liquidity را research کند یا یک basket یا market idea را بدون trading plan کند استفاده کنید. هرگز advise، order، trade، reserve یا execute نمی‌کند.
164	`ito-compute`	Query کردن live GPU inventory، submit کردن inspect authenticated Itô fixed-rate RFQ، RFQ کردن یا procurement status، revoke کردن device credentials و running explicitly gated node qualification از طریق separately installed canonical CLI. هنگامی که user می‌خواهد H100/H200 capacity پیدا کند، fixed compute rate request کند، Itô compute status check کند، GPU nodes validate کند، Itô access revoke کند یا GPU compute rent یا purchase کند استفاده کنید.
165	`ito-inference`	Inspect کردن availability of model serving روی یک completed Itô compute booking و، وقتی canonical backend available شد، hand off کردن یک explicitly confirmed serving manifest. پس از اینکه ito-compute GPU nodes booked کرده و hosted Kimi یا user OpenAI-compatible endpoint، ito-serve، self-hosted open-weights inference می‌خواهد استفاده کنید.
166	`ito-training`	Inspect کردن availability of ML training روی یک completed Itô compute booking و، وقتی canonical backend available شد، hand off کردن یک explicitly confirmed training manifest. پس از اینکه ito-compute GPU nodes booked کرده و user pre-training، fine-tuning یا RL روی آن metal می‌خواهد استفاده کنید.
167	`java-coding-standards`	استانداردهای کدنویسی Java برای Spring Boot و Quarkus services: naming، immutability، Optional usage، streams، exceptions، generics، CDI، reactive patterns و project layout. به‌صورت خودکار framework-specific conventions را apply می‌کند. هنگام نوشتن یا بازبینی Java در Spring Boot یا Quarkus service استفاده کنید.
168	`jira-integration`	هنگام retrieve کردن Jira tickets، analyzing requirements، updating ticket status، adding comments یا transitioning issues استفاده کنید. Jira API patterns از طریق MCP یا direct REST calls فراهم می‌کند.
169	`jpa-patterns`	الگوهای JPA/Hibernate برای entity design، relationships، query optimization، transactions، auditing، indexing، pagination و pooling در Spring Boot. هنگام طراحی JPA entities یا relationships یا وقتی یک Hibernate query، transaction یا N+1 problem نیاز به fixing دارد استفاده کنید.
170	`karpathy-guidelines`	Behavioral guidelines برای کاهش common LLM coding mistakes. هنگام نوشتن، بازبینی یا refactoring code برای avoid کردن overcomplication، make کردن surgical changes، surface کردن assumptions و define کردن verifiable success criteria استفاده کنید.
171	`knowledge-ops`	Knowledge base management، ingestion، sync و retrieval در multiple storage layers (local files، MCP memory، vector stores، Git repos). هنگامی که user می‌خواهد در knowledge systems خود save، organize، sync، deduplicate یا search کند استفاده کنید.
172	`kotlin-coroutines-flows`	الگوهای Kotlin Coroutines و Flow برای Android و KMP — structured concurrency، Flow operators، StateFlow، error handling و testing. هنگام نوشتن coroutines یا Flow code روی Android یا KMP یا debugging cancellation و concurrency استفاده کنید.

173	`kotlin-exposed-patterns`	الگوهای JetBrains Exposed ORM شامل DSL queries، DAO pattern، transactions، HikariCP connection pooling، Flyway migrations و repository pattern. هنگام کار با Exposed ORM — DSL یا DAO queries، transactions، یا migrations pooling استفاده کنید.
174	`kotlin-ktor-patterns`	الگوهای Ktor server شامل routing DSL، plugins، authentication، Koin DI، kotlinx.serialization، WebSockets و testApplication testing. هنگام ساخت routes، plugins، auth، DI، serialization — Ktor server یا tests استفاده کنید.
175	`kotlin-patterns`	الگوها و conventions ایدیوماتیک Kotlin، best practices برای ساخت robust، efficient و maintainable Kotlin applications با null safety، coroutines و DSL builders. هنگام نوشتن یا بازبینی Kotlin code و idiomatic structure یا null safety در question استفاده کنید.
176	`kotlin-testing`	الگوهای تست Kotlin با property-based testing، coroutine testing، MockK، Kotest و Kover coverage. TDD methodology با idiomatic Kotlin practices. هنگام نوشتن Kotlin tests با Kotest یا MockK، testing coroutines یا checking coverage استفاده کنید.
177	`kubernetes-patterns`	الگوهای workload Kubernetes، resource management، RBAC، probes، autoscaling، ConfigMap/Secret handling و kubectl debugging برای production-grade deployments. هنگام نوشتن یا بازبینی Kubernetes manifests یا debugging probes، RBAC، autoscaling یا resource limits استفاده کنید.
178	`laravel-patterns`	الگوهای معماری Laravel، routing/controllers، Eloquent ORM، service layers، queues، events، caching و API resources برای production apps. هنگام ساخت یا بازبینی Laravel apps — controllers، Eloquent، service layers، queues یا API resources استفاده کنید.
179	`laravel-plugin-discovery`	کشف و ارزیابی Laravel packages از طریق LaraPlugins.io MCP. هنگامی که user می‌خواهد plugins پیدا کند، package health check کند یا Laravel/PHP compatibility ارزیابی کند استفاده کنید.
180	`laravel-security`	Laravel security best practices — authentication، authorization، Eloquent safety، CSRF، XSS prevention، API security و secure deployment configurations. هنگام بازبینی Laravel auth، Eloquent safety، CSRF، XSS، API security یا deployment configuration استفاده کنید.
181	`laravel-tdd`	استراتژی‌های تست Laravel با PHPUnit، Pest، model factories، HTTP tests، Sanctum authentication testing، mocking و coverage. هنگام نوشتن Laravel tests با PHPUnit یا Pest یا driving یک Laravel feature test-first استفاده کنید.
182	`laravel-verification`	Verification loop برای Laravel projects: env checks، linting، static analysis، tests با coverage، security scans و deployment readiness. هنگام verify کردن Laravel project قبل از merge یا deploy استفاده کنید.
183	`latency-critical-systems`	بهینه‌سازی و تأیید سیستم‌های latency-sensitive — realtime dashboards، market data feeds، streaming agents، execution gateways، queues و caches — با tracking p50/p95/p99 latency، freshness age و queue depth، mapping hot paths و running live readbacks. هنگامی که throughput، p95 latency یا data freshness مهم است استفاده کنید.
184	`lead-intelligence`	AI-native lead intelligence و outreach pipeline. جایگزین Apollo، Clay و ZoomInfo با agent-powered signal scoring، mutual ranking، warm path discovery، source-derived voice modeling و channel-specific outreach در LinkedIn، email و X. هنگامی که کاربر می‌خواهد high-value contacts را find، qualify و reach کند استفاده کنید.
185	`liquid-glass-design`	iOS 26 Liquid Glass design system — dynamic glass material با blur، reflection و interactive morphing برای SwiftUI، UIKit و WidgetKit. هنگام ساخت iOS 26 Liquid Glass UI در SwiftUI، UIKit یا WidgetKit استفاده کنید.
186	`living-docs-governance`	جلوگیری از rot شدن مستندات project بلندمدت با assign کردن existing project docs به roles روشن، constitution، status map، history، سپس wiring active agent harness به those canonical sources. در maintain phase هنگامی که docs از code drift می‌کنند، agents context را بین sessions از دست می‌دهند یا intentional removals دوباره recreate می‌شوند استفاده کنید.
187	`llm-trading-agent-security`	Security patterns برای autonomous trading agents با wallet یا transaction authority. پوشش prompt injection، spend limits، pre-send simulation، circuit breakers، MEV protection و key handling. هنگامی که یک

		autonomous agent wallet یا transaction authority دارد و limits، simulation یا key handling آن نیاز به review دارد استفاده کنید.
188	`logistics-exception-management`	expertise codified freight-exception handling برای shipment delays، damages، losses، shortages و carrier disputes، با escalation protocols، carrier-specific behaviors by mode، claims procedures و eat-the-cost vs fight-the-claim judgment frameworks. هنگام handling shipping exceptions، freight claims، delivery issues یا carrier disputes استفاده کنید.
189	`loop-design-check`	طراحی یک goal-oriented agent loop یا review آن برای failure modes: spinning، Goodhart-gaming verifier یا running a wrong answer to completion. پوشش machine-decidable goals، loop types، plan/build/judge skeletons و runaway prevention. هنگام designing، writing یا checking یک agent loop استفاده کنید.
190	`mailtrap-email-integration`	راهنمای agentها برای integrating transactional email sending از طریق Mailtrap's Email API، شامل sandbox testing، domain verification و API authentication. هنگام implement کردن email-sending features، debugging delivery issues یا setting up safe dev/staging email testing استفاده کنید.
191	`make-interfaces-feel-better`	اعمال concrete design-engineering details که interfaces را polished احساس می‌کنند. هنگام review یا improve کردن UI spacing، typography، borders، shadows، motion، hit areas، icons، text wrapping و interaction states استفاده کنید.
192	`manim-video`	ساخت reusable Manim explainers برای technical concepts، graphs، system diagrams و product walkthroughs، سپس hand off به wider ECC video stack در صورت نیاز. هنگامی که کاربر یک clean animated explainer (نه generic talking-head script) می‌خواهد استفاده کنید.
193	`market-research`	انجام market research، competitive analysis، investor due diligence و industry intelligence با source attribution و decision-oriented summaries. هنگامی که کاربر fund، competitor comparisons، market sizing، technology scans، research یا research که business decisions را inform می‌کند می‌خواهد استفاده کنید.
194	`marketing-campaign`	End-to-end marketing campaign planning و execution. پوشش campaign، positioning، audience research، short-form video scripts، ad copy، email sequences، social posts، landing page copy، angle definition و content calendars. به‌عنوان orchestration layer برای multi-channel product launches استفاده کنید.
195	`master-agreement-generator`	تولید review drafts of counterparty master agreements از یک template به‌علاوهٔ یک JSON spec، با role-selected clauses و Schedule A workflow محدود به notice authority of executed agreement. هنگامی که نیاز به reproducible drafting و separately reviewed execution preparation دارید استفاده کنید.
196	`mcp-dependency-review`	Statically review کردن MCP configuration برای mutable package references قبل از approval یا CI، بدون executing discovered MCP servers. هنگام Windsurf، VS Code، Claude، Cursor، mcp.json یا سایر reviewing repo-scoped MCP config استفاده کنید.
197	`mcp-server-patterns`	ساخت MCP servers با Node/TypeScript SDK — tools، resources، prompts، Zod validation، stdio vs Streamable HTTP. استفاده از Context7 یا official MCP docs برای latest API. هنگام building یا debugging یک MCP server — tools، resources، prompts، validation یا transport choice استفاده کنید.
198	`memory-bank`	استفاده در شروع هر session و هرگاه دربارهٔ 'memory bank'، remembering past project context، recalling یا previously dictated message texts saving new user-dictated texts so they survive session close پرسیده شد. persistent Memory Bank در ~/./config/opencode/memory/ را که per-project history و هر user-dictated message با SHA-256 identity ذخیره می‌کند بارگذاری می‌کند.
199	`messages-ops`	گردش‌کار live messaging مبتنی بر evidence برای ECC. هنگامی که کاربر می‌خواهد texts یا DMها را read کند، یک recent one-time code recover کند، یک thread را قبل از replying inspect کند یا prove کند کدام message source واقعاً checked شد استفاده کنید.
200	`ml-adoption-playbook`	methodology end-to-end برای AI agents و software engineers برای افزودن machine learning algorithms به existing non-ML codebases. پوشش architectural decoupling، data readiness، problem framing و baseline model integration. هنگام افزودن یک machine learning capability به codebase که هیچ ندارد، از problem framing تا baseline model استفاده کنید.

201	`mle-workflow`	گردش‌کار production machine-learning engineering برای data contracts، reproducible training، model evaluation، deployment، monitoring و rollback. هنگام ساخت، بازبینی یا hardening ML systems فراتر از one-off notebooks استفاده کنید.
202	`module-regression`	Govern کردن cross-module regression در large repositories با یک REGRESSION.md ledger که downstream consumers هر module و executable acceptance command را record می‌کند. پس از یک change، changed module و هر affected downstream command را run کنید و observed exit codes را به‌عنوان regression evidence تحت delivery policy پروژه استفاده کنید.
203	`motion-advanced`	الگوهای motion پیشرفته برای React/Next.js — drag & drop، gestures، text animations، SVG path drawing، custom hooks، imperative sequences (useAnimate)، loaders و full API decision tree. نیازمند motion-foundations. هنگام ساخت drag and drop، gestures، text یا SVG animation یا imperative animation sequences در React یا Next.js استفاده کنید.
204	`motion-foundations`	Motion tokens، spring presets، performance rules، device adaptation، accessibility enforcement و SSR safety برای React/Next.js با استفاده از motion/react. Foundation layer — همهٔ motion skills دیگر به این وابسته‌اند. هنگام setup motion tokens، spring presets، reduced-motion handling یا SSR-safe animation در React یا Next.js استفاده کنید.
205	`motion-patterns`	الگوهای animation production-ready برای React/Next.js — button، modal، toast، stagger، page transitions، exit animations، scroll و layout — built on motion-foundations tokens و springs. هنگام animate کردن یک UI element خاص در React یا Next.js استفاده کنید.
206	`mybatis-patterns`	الگوهای MyBatis و MyBatis-Spring برای XML mapper design، dynamic و annotation SQL، result mapping، pagination، batching، transactions، SQL safety و query performance. هنگام ساخت یا بازبینی Java persistence code با MyBatis، Spring Boot، MyBatis-Spring یا MyBatis-based legacy applications استفاده کنید.
207	`mysql-patterns`	الگوهای MySQL و MariaDB schema، query، indexing، transaction، replication و connection-pool برای production backends. هنگام طراحی MySQL یا MariaDB schemas و indexes یا وقتی یک query، transaction یا replica lags استفاده کنید.
208	`nanoclaw-repl`	عملیات و extend کردن NanoClaw، ECC's zero-dependency session-aware REPL، با persistent markdown-backed sessions و slash commands برای cross-session branching، session loading، model switching، history compaction، search و export. هنگام running یا extending scripts/claw.js یا branching، resuming، searching، compacting یا exporting یک NanoClaw session استفاده کنید.
209	`nasiko-control-plane`	مدیریت experimental Nasiko CLI lifecycle از طریق ECC — read-only status checks، consent-gated install of pinned qualified version با dry-run preview و ownership-checked uninstall، تحت explicit telemetry و secrets boundaries. هنگامی که user می‌خواهد Nasiko CLI را inspect، install یا remove کند یا check کند آیا present است استفاده کنید.
210	`nestjs-patterns`	الگوهای معماری NestJS برای modules، controllers، providers، DTO validation، guards، interceptors، config و production-grade TypeScript backends. هنگام ساخت یا بازبینی NestJS backend — modules، providers، DTO validation، guards یا interceptors استفاده کنید.
211	`netmiko-ssh-automation`	الگوهای safe Python Netmiko برای read-only collection، bounded batch SSH، TextFSM parsing، guarded config changes، timeouts و network automation error handling. هنگام automate کردن network device access با Python Netmiko، چه collecting state چه pushing guarded config changes استفاده کنید.
212	`network-bgp-diagnostics`	الگوهای diagnostics-only BGP troubleshooting برای neighbor state، route exchange، prefix policy، AS path و inspection و safe evidence collection. هنگامی که BGP neighbor down است، routes missing هستند یا prefix policy و AS path نیاز به inspection دارند استفاده کنید.
213	`network-config-validation`	Pre-deployment checks برای router و switch configuration، شامل dangerous commands، duplicate addresses، subnet overlaps، stale references، management-plane risk و IOS-style security hygiene. هنگام review کردن یک router یا switch configuration قبل از deployment استفاده کنید.

214	`network-interface-health`	تشخیص interface errors، drops، CRCs، duplex mismatches، flapping، speed negotiation issues و counter trends روی routers، switches و Linux hosts. هنگامی که یک interface errors، drops، CRCs، flapping یا duplex/speed mismatch نشان می‌دهد استفاده کنید.
215	`nextjs-turbopack`	راهنمای Next.js 16+ و Turbopack — incremental Rust bundling، file-system caching، faster dev startup و HMR، Turbopack vs webpack tradeoffs و تغییر نام middleware.ts به proxy.ts. هنگام developing یا debugging Next.js 16+ apps، diagnosing slow dev startup یا hot reload، choosing بین bundlers یا reviewing middleware/proxy file naming استفاده کنید.
216	`nodejs-keccak256`	جلوگیری از Ethereum hashing bugs در JavaScript و NIST SHA3 sha3-256. Node's است، نه TypeScript. Ethereum Keccak-256، و silently selectors، signatures، storage slots و address derivation را break می‌کند. هنگام hashing برای Ethereum در JavaScript یا TypeScript یا وقتی یک storage slot، signature، selector یا derived address اشتباه است استفاده کنید.
217	`nutrient-document-processing`	پردازش، تبدیل، sign، redact، extract، OCR و fill کردن documents با استفاده از Nutrient DWS API. کار با PDFs، DOCX، XLSX، PPTX، HTML و images. هنگام signing، redacting، extracting from، converting یا filling documents از طریق Nutrient DWS API استفاده کنید.
218	`nuxt4-patterns`	الگوهای Nuxt 4 app برای lazy loading، route rules، performance، hydration safety و SSR-safe data fetching با useFetch و useAsyncData. هنگام ساخت یا بازبینی Nuxt 4 app یا hydration mismatches و debugging و SSR-safe data fetching استفاده کنید.
219	`open-bale`	هنگامی که user می‌خواهد "messenger "Bale (بله) را open یا launch کند، یا یک contact/chat پیدا کند و message در Bale روی Windows بفرستد. Bale Web Chrome app window را باز می‌کند و از طریق low-level SendInput drive می‌کند چون synthetic key events به Chrome نمی‌رسند.
220	`open-eitaa`	هنگامی که user می‌خواهد "messenger "Eitaa (ایتا) را open یا launch کند، یا یک contact/chat پیدا کند و message در Eitaa روی Windows بفرستد. Eitaa Desktop app را باز می‌کند و از طریق UIA/OCR به‌علاوهٔ low-level SendInput drive می‌کند.
221	`open-rubika`	هنگامی که user می‌خواهد "messenger "Rubika (روبیکا) را open یا launch کند، یا یک contact/chat پیدا کند و message در Rubika روی Windows بفرستد. Rubika Web Chrome app window را باز می‌کند و از طریق low-level SendInput drive می‌کند. همچنین approach سریع‌تر (rubpy) API-based را وقتی UI automation خیلی کند است پوشش می‌دهد.
222	`open-soroush`	هنگامی که user می‌خواهد "messenger "Soroush Plus را open یا launch کند و message بفرستد. روش مشابه Bale/Rubika با Chrome app window و SendInput.
223	`openclaw-persona-forge`	ساخت یک complete lobster persona برای OpenClaw AI Agent — identity، SOUL.md، role-based boundary rules، name و avatar prompt. بر اساس ترجیحات user یا random draw. هنگامی که user می‌خواهد یک OpenClaw lobster persona ایجاد، design یا customize کند استفاده کنید.
224	`opensource-pipeline`	Open-source pipeline: fork، sanitize و package کردن private projects برای agent (forker، sanitizer، packager) safe public release. ۳ chain را می‌کند. Triggers: '/opensource'، 'open source this'، 'make this public'، 'prepare for open source'. هنگامی که یک private project باید fork، stripped of secrets و packaged برای public release شود استفاده کنید.
225	`operator-approval-loop`	Operator approval contract با internal filing notices برای agent-drafted outbound messages، hashed drafts، epoch-keyed decisions، durable delivery claims و receipts و pre-draft baseline gate. هنگامی که یک agent به external counterparties draft می‌کند و یک human operator باید هر send را قبل از leaving approve، reject یا steer کند استفاده کنید.
226	`orch-add-feature`	هماهنگی ساخت یک brand-new feature end to end — research، plan، TDD implementation، review و gated commit — با delegate کردن هر phase به matching ECC agent. هنگام افزودن یک capability که هنوز وجود ندارد استفاده کنید.
227	`orch-build-mvp`	هماهنگی bootstrapping یک working MVP از یک design یا spec document — ingest SDD/PRD، plan thin vertical slices، scaffold اولین end-to-end slice، سپس drive کردن generator-evaluator build loop با review و

		gated feat commits. هنگامی که یک design یا spec document باید به یک running MVP تبدیل شود استفاده کنید.
228	`orch-change-feature`	هماهنگی تغییر یک existing، working feature به update — new desired behavior کردن tests به new spec، تغییر implementation برای match، review و gated commit. هنگامی که behavior broken نیست اما باید different باشد استفاده کنید.
229	`orch-fix-defect`	هماهنگی fix کردن یک bug — reproduce کردن آن به‌عنوان یک failing regression test، fix به green، review و gated commit — با delegate کردن هر phase به matching ECC agent. هنگامی که existing behavior broken یا wrong است استفاده کنید.
230	`orch-pipeline`	Shared orchestration engine پشت orch-* skill family — gated Research-Plan-TDD-Review-Commit pipeline، size classifier، agent command map و two human gates (plan approval، commit confirmation) که orch-* operation skills به آنها delegate می‌کنند. به‌صورت indirect از طریق orch-add-feature، orch-fix-defect، orch-change-feature، orch-refine-code یا orch-build-mvp استفاده کنید.
231	`orch-refine-code`	هماهنگی یک behavior-preserving refactor — تأیید green بودن tests، restructure بدون تغییر behavior، green نگه داشتن tests، review و gated commit. هنگامی که structure باید improve شود اما behavior نباید change شود استفاده کنید.
232	`osint-investigation`	methodology OSINT با sparse-clue برای extract کردن connect، overlooked leads کردن fragmented evidence و multi-step CTF challenges، image/video geolocation، event برای testing explanations across sources. reconstruction، public-account و infrastructure research، artifact interpretation، entity verification یا stalled investigations استفاده کنید.
233	`parallel-execution-optimizer`	سریع‌تر کردن یک task با تبدیل آن به dependency graph of parallel lanes با batched reads، lane matrix و checks، write surfaces isolated by file، worktree، branch یا service و final verification table. هنگامی که user می‌خواهد یک task بسیار سریع‌تر از طریق isolated worktrees، batched tool calls، concurrent agents، parallel work یا many independent verification lanes بدون از دست دادن correctness انجام شود استفاده کنید.
234	`perl-patterns`	ایدیوم‌های best practices، modern Perl 5.36+ و conventions برای ساخت maintainable، robust Perl applications. هنگام نوشتن یا بازبینی modern Perl 5.36+ code استفاده کنید.
235	`perl-security`	Security جامع Perl پوشش‌دهندهٔ taint mode، input validation، safe process execution، DBI parameterized queries، web security (XSS/SQLi/CSRF) و perlcritic security policies. هنگام بازبینی Perl input handling، process execution، DBI queries یا web-facing code استفاده کنید.
236	`perl-testing`	الگوهای تست Perl با Test2::V0، Test::More، prove runner، mocking، coverage و Devel::Cover با TDD methodology. هنگام نوشتن Perl tests با Test2::V0 یا Test::More یا measuring coverage استفاده کنید.
237	`plan-canvas`	باز کردن plans و HTML artifacts در یک local browser canvas که human elements را annotate می‌کند، chat می‌کند و approve یا request changes می‌کند بدون leaving the page. هنگام presenting یک plan برای review یا وقتی feedback مانند 'move this، change that' easier pointed at than typed' است استفاده کنید.
238	`plan-orchestrate`	خواندن یک plan document، decompose کردن آن به steps، classify کردن هر step و emit کردن ready-to-paste ECC slash command برای آن step، با carry کردن step identifier و scope در command's own documented argument form. فقط generative — هرگز commands را invoke نمی‌کند. هنگامی که user یک multi-step plan دارد و می‌خواهد right command برای هر step بدون picking by hand داشته باشد استفاده کنید.
239	`plankton-code-quality`	Write-time code quality enforcement با استفاده از auto-formatting، linting — Plankton و Claude-powered fixes روی هر file edit از طریق hooks. هنگام setup کردن write-time formatting، linting یا auto-fix hooks روی file edits استفاده کنید.
240	`postgres-patterns`	الگوهای PostgreSQL database برای indexing، schema design، query optimization و security. بر اساس Supabase best practices. هنگام طراحی PostgreSQL schemas، indexes یا RLS policies یا وقتی یک query خیلی کند است استفاده کنید.

241	`prediction-market-oracle-research`	Research کردن prediction markets به‌عنوان data sources یا oracle signals برای dashboards، agents، products و source-grounded analysis of market-implied probabilities، corporate decision intelligence. استفاده برای caveats و integration patterns بدون investment advice. هنگام evaluate کردن prediction markets به‌عنوان data source یا oracle signal برای یک product، agent یا dashboard استفاده کنید.
242	`prediction-market-risk-review`	Review کردن prediction-market، basket، oracle و trading-agent workflows برای compliance، safety، data-quality، privacy و execution risk. قبل از اینکه هر API keys، user portfolio data، venue auth یا workflow trade را handle کند استفاده کنید.
243	`prisma-patterns`	الگوهای Prisma ORM برای TypeScript backends — schema design، query optimization، transactions، pagination و critical traps مانند updateMany returning count not records، $transaction timeouts، migrate dev resetting DB، @updatedAt skipped on bulk writes و serverless connection exhaustion. هنگام نوشتن Prisma schema یا query یا debugging transactions، migrations یا serverless connection limits استفاده کنید.
244	`product-capability`	Translate کردن PRD intent، roadmap asks یا product discussions به یک implementation-ready capability plan که interfaces، invariants، constraints و unresolved decisions را قبل از multi-service work expose می‌کند. هنگامی که user نیاز به ECC-native PRD-to-SRS lane (نه vague planning prose) دارد استفاده کنید.
245	`product-lens`	Validate کردن the why قبل از building از طریق four product diagnostics — یک YC-style product diagnostic که go/no-go recommendation با PRODUCT-BRIEF.md تولید می‌کند، یک product-market-fit signals، یک user journey audit measuring time-to-value و ICE feature prioritization. هنگام pressure-testing product direction، choosing بین features، sanity-checking یک launch یا converting یک vague idea به product brief استفاده کنید.
246	`production-audit`	Local-evidence production readiness audit برای pre-launch reviews، post-merge checks و shipped apps، بدون sending repo data به external audit service. هنگام auditing production 'what breaks in prod?' questions' قبل از launch، پس از merge یا وقتی پرسیده می‌شود what breaks in prod استفاده کنید.
247	`production-scheduling`	expertise codified برای production scheduling، job sequencing، line balancing، changeover optimization و bottleneck resolution در discrete و batch manufacturing. Informed by production schedulers با ۱۵+ years experience. شامل TOC/drum-buffer-rope، SMED، OEE analysis، disruption response frameworks و ERP/MES interaction patterns. هنگام scheduling production، resolving bottlenecks، optimizing changeovers یا responding to disruptions balancing manufacturing lines استفاده کنید.
248	`project-flow-ops`	عملیات execution flow در GitHub و Linear با triaging issues و linking active work، pull requests و keeping GitHub public-facing در حالی که Linear internal execution layer می‌ماند. هنگامی که PR، user backlog control یا triage GitHub-to-Linear coordination می‌خواهد استفاده کنید.
249	`prompt-caching-strategy`	بهینه‌سازی LLM prompt caching hit rate برای کاهش API costs و بهبود latency. prompt structure را analyze می‌کند، cacheable vs non-cacheable content را شناسایی می‌کند و restructuring برای maximize کردن cache reuse پیشنهاد می‌دهد. TRIGGER هنگامی که user می‌گوید 'reduce LLM cost'، 'cache hit rate'، 'prompt cache'، 'optimize token usage'، 'cheaper LLM calls'' یا دربارهٔ prompt caching strategies برای OpenAI/Anthropic/other providers می‌پرسد.
250	`prompt-optimizer`	تحلیل draft prompts، detect کردن intent و missing context، match کردن skills، ECC commands و agents و output یک ready-to-paste optimized prompt با diagnosis و rationale — فقط advisory، هرگز task را execute نمی‌کند. هنگامی که user می‌گوید 'help me'، 'rewrite this prompt'، 'improve my prompt'، 'optimize prompt' یا 帮我优化这个指令 استفاده کنید.
251	`python-patterns`	ایدیوم‌های type hints، PEP 8 standards، Pythonic و best practices برای ساخت robust، efficient و maintainable Python applications. هنگام نوشتن یا بازبینی Python code، typing، idiomatic structure یا PEP 8 در question استفاده کنید.
252	`python-testing`	استراتژی‌های تست Python با استفاده از parametrization، mocking، fixtures، TDD methodology، pytest و coverage requirements. هنگام نوشتن fixtures، mocks، parametrization — pytest tests یا coverage استفاده کنید.

253	`pytorch-patterns`	الگوها و best practices PyTorch deep learning برای ساخت robust، efficient، و reproducible training pipelines، model architectures و data loading. هنگام نوشتن یا بازبینی PyTorch training loops، model architectures یا data loading یا وقتی یک run reproduce نمی‌شود استفاده کنید.
254	`quality-nonconformance`	Quality control و non-conformance management برای IATF 16949، AS9100): NCR lifecycle (FDA 21 CFR 820، disposition، 5-Why/Ishikawa/fault-tree/8D root cause analysis، CAPA و systems، SPC interpretation، AQL sampling و supplier quality audits. هنگام investigating non-conformances، handling supplier quality issues یا performing root cause analysis، managing CAPAs، interpreting SPC data استفاده کنید.
255	`quarkus-patterns`	الگوهای معماری Quarkus 3.x LTS با Camel برای data access، CDI services، RESTful API design، messaging با Panache و async processing. برای Java Quarkus backend work با event-driven architectures. هنگام ساخت یا بازبینی Quarkus service، به‌ویژه با Camel messaging یا Panache data access استفاده کنید.
256	`quarkus-security`	الگوهای پیاده‌سازی security Quarkus: JWT و OIDC authentication، @RolesAllowed RBAC و SecurityIdentity، Bean Validation و checks، BCrypt password hashing، custom validators، parameterized Panache queries، CORS و Vault، audit logging، rate limiting، security headers یا environment-variable secrets و dependency CVE scanning. هنگام افزودن authentication یا managing secrets، validating input، authorization یا hardening Quarkus application استفاده کنید.
257	`quarkus-tdd`	Test-driven development برای Quarkus 3.x LTS با استفاده از Camel testing، REST Assured، Mockito، JUnit 5 و JaCoCo. هنگام افزودن features، fixing bugs یا refactoring event-driven services استفاده کنید.
258	`quarkus-verification`	Verification loop برای Quarkus projects: build، static analysis (Checkstyle، PMD، SpotBugs)، tests با JaCoCo، coverage، OWASP dependency و container security scans، GraalVM native compilation، health checks و config validation. هنگام verifying Quarkus service قبل از PR، پس از major refactoring یا dependency upgrades یا pre-deploy استفاده کنید.
259	`rails-patterns`	الگوهای framework Ruby on Rails برای Rails 7.1+ و Active Storage.x apps.8، پوشش directory contract، skinny controllers، service objects، form objects، query objects، idiomatic ActiveRecord، background jobs، ViewComponent، Hotwire و Rails 8 Solid stack. هنگام ساخت یا بازبینی Rails apps، controllers، models، services، jobs یا views استفاده کنید.
260	`ralphinho-rfc-pipeline`	Split کردن یک RFC به یک multi-agent execution DAG — decompose به work units با dependencies و acceptance tests، run research، plan، implement، test و review per unit، سپس merge از طریق queue با re-based branches و final system verification. هنگامی که یک feature برای یک single agent pass خیلی بزرگ است، یا orchestrating RFC-driven multi-agent execution یا managing merge queues across agent-built units استفاده کنید.
261	`react-native-patterns`	الگوهای React Native و Expo app — Expo Router navigation، state separation (server/client/route/form)، TanStack Query data fetching با native APIs، NativeWind/StyleSheet styling، performant lists، Zod و secure storage. هنگام ساخت یا edit کردن navigation، React Native/Expo screens، components یا data layers استفاده کنید.
262	`react-patterns`	الگوهای React 18/19 شامل Suspense + error boundaries، server/client component boundaries، hooks discipline، state management decision trees، data fetching، form actions، boundaries و accessibility-first composition. هنگام نوشتن یا بازبینی React components استفاده کنید.
263	`react-performance`	الگوهای بهینه‌سازی performance React و Next.js adapted از Vercel Engineering's React Best Practices. organize کردن ۷۰+ rules در ۸ priority category — waterfalls، bundle size، server-side، client fetching، re-render، rendering، JS micro-perf، advanced. هنگام نوشتن، بازبینی یا refactoring React/Next.js code برای performance استفاده کنید.
264	`react-testing`	تست component React با React Testing Library، Vitest/Jest، MSW برای accessibility، network mocking، assertions با axe و decision boundary بین component tests و end-to-end runs Playwright/Cypress. هنگام نوشتن یا fixing tests برای React components، hooks یا pages استفاده کنید.

265	`recsys-pipeline-architect`	طراحی composable recommendation، ranking و feed pipelines با استفاده از six-stage Source→Hydrator→Filter→Scorer→Selector→SideEffect framework popularized by xAI's open-sourced For You algorithm. هر زمان که user در حال ساخت systemی است که 'top K items for a (user, context)' را pick می‌کند — social feeds، content CMSs، RAG rerankers، task prioritizers، notification triage، search reranking، ad ranking استفاده کنید.
266	`recursive-decision-ledger`	اجرای repeated rollouts ('Prime Gauss' style recursive prompting) در حالی که یک append-only decision ledger از trials، marks، coherence checks و promotion gates نگه می‌دارد، به‌طوری که recursive confidence هرگز live trading، deploy یا destructive actions را auto-approve نکند. هنگامی که user repeated rollouts، marked decision processes، high-dimensional search، stochastic optimization، local-optima exploration، ensemble comparison یا recursive reasoning با visible evidence trail می‌خواهد استفاده کنید.
267	`redis-patterns`	الگوهای pub/sub، Redis data structure، caching strategies، distributed locks، rate limiting و connection management برای production applications. هنگام افزودن rate limiting، distributed lock، caching یا pub/sub با Redis یا وقتی key design نیاز به review دارد استفاده کنید.
268	`regex-vs-llm-structured-text`	Decision framework برای parsing structured text (quizzes، forms، invoices، receipts، tables) با یک hybrid regex-first pipeline — regex extraction ۹۵٪+ را cheaply handle می‌کند، یک low-confidence scorer items را flag می‌کند و یک LLM validator فقط edge cases را fix می‌کند. هنگام choosing بین regex و LLM برای text extraction، ساخت یک cheap document parser یا optimizing extraction cost و accuracy استفاده کنید.
269	`remotion-video-creation`	Best practices برای Remotion - Video creation in React. ۲۹ domain-specific rules پوشش‌دهندهٔ 3D، animations، audio، captions، charts، transitions و بیشتر. هنگام building video in React با Remotion — animations، audio، captions، charts یا transitions استفاده کنید.
270	`repo-scan`	Bootstrap pointer که external repo-scan skill را از یک pinned، reviewable commit install می‌کند. هنگامی که repo-scan باید قبل از running its cross-stack source-code asset audit installed شود؛ این ECC pointer خود audit را perform نمی‌کند.
271	`research-ops`	گردش‌کار research مبتنی بر evidence برای ECC. هنگامی که کاربر fresh facts، comparisons، enrichment یا recommendation built from current public evidence و any supplied local context می‌خواهد استفاده کنید.
272	`returns-reverse-logistics`	expertise codified برای returns authorization، receipt، inspection، disposition decisions، refund processing، fraud detection و warranty claims management. Informed by returns operations managers با ۱۵+ years experience. شامل disposition economics، fraud pattern recognition، grading frameworks و vendor recovery processes. هنگام handling product returns، reverse logistics، refund decisions، return fraud detection یا warranty claims استفاده کنید.
273	`rules-distill`	Scan کردن skills برای extract کردن cross-cutting principles و distill کردن آنها به append، revise یا rules — new rule files. هنگامی که same principle keeps recurring across skills و belongs in یک rule file استفاده کنید.
274	`rust-patterns`	الگوهای ایدیوماتیک Rust، ownership، error handling، traits، concurrency و best practices برای ساخت safe، performant applications. هنگام نوشتن یا بازبینی Rust code و traits ،error handling ،ownership یا concurrency در question استفاده کنید.
275	`rust-testing`	الگوهای تست Rust شامل unit tests، integration tests، async testing، property-based testing، mocking و coverage. TDD methodology را دنبال می‌کند. هنگام نوشتن Rust tests — unit، integration، async، property-based یا coverage استفاده کنید.
276	`safety-guard`	Guard در برابر destructive operations با three modes: Careful dangerous commands (rm -rf، git push --force، DROP TABLE) را برای confirmation intercept می‌کند، Freeze writes را به یک directory lock می‌کند و Guard both را از طریق PreToolUse hooks combine می‌کند. هنگام کار روی production systems، running agents به‌صورت autonomously، restricting edits به یک directory یا during migrations، deploys و data changes استفاده کنید.
277	`santa-method`	Multi-agent adversarial verification: two independent reviewers با same rubric باید both pass کنند قبل از اینکه output ship شود، با fix-and-re-review convergence loop و human escalation cap. هنگام gating publishing، production deploys، compliance یا brand-sensitive content یا hallucination-prone claims قبل از ship استفاده کنید.

278	`scientific-db-pubmed-database`	گردش‌کار direct PubMed و NCBI E-utilities search برای biomedical literature، MeSH queries، PMID lookup، citation retrieval و API-backed literature monitoring. هنگامی که task نیاز به biomedical literature از PubMed (نه general web search) دارد استفاده کنید.
279	`scientific-db-uspto-database`	گردش‌کار USPTO patent و trademark data برای official record lookup، PatentSearch queries، TSDR checks، assignment data و reproducible IP research logs. هنگامی که task نیاز به official United States patent یا trademark records از USPTO systems دارد استفاده کنید.
280	`scientific-pkg-gget`	گردش‌کار gget CLI و Python برای quick genomic database queries، sequence lookup، BLAST-style searches، enrichment checks و reproducible bioinformatics evidence logs. هنگامی که task نیاز به quick bioinformatics lookup در genomic reference databases با gget CLI یا Python package دارد استفاده کنید.
281	`scientific-thinking-literature-review`	گردش‌کار systematic literature-review برای academic، biomedical، technical و scientific topics، شامل search، citation checks، synthesis، source screening، planning و evidence logging. هنگامی که task find، screen، synthesize و cite کردن یک body of academic یا technical literature است استفاده کنید.
282	`scientific-thinking-scholar-evaluation`	ارزیابی structured scholarly-work برای papers، proposals، literature reviews، methods sections، evidence quality، citation support و research-writing feedback. هنگام evaluate کردن academic یا scientific work — papers، proposals، methods sections یا evidence quality — در برابر یک repeatable rubric استفاده کنید.
283	`search-first`	گردش‌کار research-before-coding: search کردن npm/PyPI، MCP servers، skills و GitHub برای existing tools قبل از نوشتن custom code، سپس adopt، extend یا build. researcher agent را برای non-trivial needs launch می‌کند. هنگام شروع یک feature، افزودن یک dependency یا integration یا وقتی در حال نوشتن utility‌ای هستید که ممکن است قبلاً وجود داشته باشد استفاده کنید.
284	`security-bounty-hunter`	Hunt کردن برای exploitable، bounty-worthy security issues در repositories. تمرکز روی remotely reachable vulnerabilities که qualify برای real reports می‌شوند به جای noisy local-only findings. هنگام hunting reportable، remotely reachable vulnerabilities در یک repository استفاده کنید.
285	`security-review`	هنگام افزودن authentication، handling user input، کار با secrets، ایجاد API endpoints یا implement کردن payment/sensitive features استفاده کنید. comprehensive security checklist و patterns فراهم می‌کند.
286	`security-scan`	Scan کردن Claude Code configuration (.claude/ directory) برای security vulnerabilities، misconfigurations و injection risks با استفاده از AgentShield. CLAUDE.md، settings.json، MCP servers، hooks و agent definitions را check می‌کند. هنگام auditing یک .claude/ directory استفاده کنید.
287	`seo`	Audit، plan و implement کردن SEO improvements در technical SEO، on-page optimization، structured data، Core Web Vitals و content strategy. هنگامی که schema، SEO remediation، user better search visibility، markup، sitemap/robots work یا keyword mapping می‌خواهد استفاده کنید.
288	`skill-comply`	Visualize کردن whether skills، rules و agent definitions واقعاً followed می‌شوند — auto-generate کردن scenarios در ۳ prompt strictness level، running agents، classify کردن behavioral sequences و report کردن compliance rates با full tool call timelines. هنگام check کردن whether agents واقعاً skills، rules و definitions داده‌شده را follow می‌کنند (نه assuming) استفاده کنید.
289	`skill-host-compat`	Lint کردن ECC skills برای Codex/Cursor-safe frontmatter و Claude-only substitutions قبل از یک skill PR.
290	`skill-scout`	Search کردن GitHub، marketplace، existing local و web skill sources قبل از creating یک new skill. هنگامی که user می‌خواهد یک skill برای یک workflow create، build، fork یا find کند استفاده کنید.
291	`skill-stocktake`	هنگام auditing Claude skills و commands برای quality استفاده کنید. Quick Scan (changed skills only) و Full Stocktake modes با sequential subagent batch evaluation را پشتیبانی می‌کند.
292	`social-graph-ranker`	Weighted social-graph ranking برای warm intro discovery، bridge scoring و network gap analysis در X و LinkedIn. هنگامی که user reusable graph-ranking engine itself (نه broader outreach یا network-maintenance workflow layered on top) می‌خواهد استفاده کنید.

293	`social-messaging`	هنگامی که user می‌خواهد messages روی social messengers (Bale، Rubika، Eitaa، Soroush Plus) بفرستد یا هنگام dealing با recipient contacts/search/find/verify قبل از sending استفاده کنید. برای methodology strict، verified messaging و contact management روی Iranian messengers از Windows فراهم می‌کند.
294	`social-publisher`	Agent-driven scheduling و publishing social media posts در ۱۳ platform از طریق SocialClaw. هنگامی که user می‌خواهد به X، LinkedIn، Instagram، Facebook Pages، TikTok، Discord، Telegram، YouTube، Reddit، WordPress یا Pinterest publish کند یا media upload، campaigns manage یا post delivery status monitor کند استفاده کنید.
295	`springboot-patterns`	الگوهای معماری Spring Boot، REST API design، layered services، data access، caching، async processing و logging. برای Java Spring Boot backend work. هنگام ساخت یا بازبینی Spring Boot backend — REST layer، services، data access، caching یا async work استفاده کنید.
296	`springboot-security`	Spring Security best practices برای authn/authz، validation، CSRF، secrets، headers، rate limiting و dependency security در Java Spring Boot services. هنگام بازبینی Spring Security authn/authz، validation، CSRF، secrets، headers یا rate limiting استفاده کنید.
297	`springboot-tdd`	Test-driven development برای Spring Boot با استفاده از Testcontainers، MockMvc، Mockito، JUnit 5 و JaCoCo. هنگام افزودن features، fixing bugs یا refactoring استفاده کنید.
298	`springboot-verification`	اجرای Maven — full Spring Boot verification loop یا Gradle build، SpotBugs، PMD و Checkstyle static analysis، unit و Testcontainers integration tests با JaCoCo coverage، OWASP dependency و secret scans و diff review — تولید pass/fail readiness report. هنگام آماده‌سازی Spring Boot pull request، validation coverage thresholds یا running pre-deploy verification استفاده کنید.
299	`story-lifecycle`	مدیریت یک file-based epic → story → sprint development loop بدون نیاز به external project management tool. epics را create و track می‌کند، آنها را به user stories break می‌کند، به sprints assign می‌کند و implementation را story by story drive می‌کند. هنگام شروع یک feature initiative جدید یا وقتی team Jira/Linear/Taiga ندارد اما structured delivery tracking نیاز دارد استفاده کنید.
300	`strategic-compact`	پیشنهاد manual context compaction در logical intervals برای preserve کردن context از طریق task phases به جای arbitrary auto-compaction. هنگامی که یک session به context limit نزدیک می‌شود و یک task phase natural برای compact است استفاده کنید.
301	`swift-actor-persistence`	Thread-safe data persistence در Swift با استفاده از in-memory cache — actors با file-backed storage، eliminating data races by design. هنگام persist کردن data در Swift و یک data race یا thread-safety problem نیاز به designing out دارد استفاده کنید.
302	`swift-concurrency-6-2`	Swift 6.2 Approachable Concurrency — single-threaded by default، @concurrent برای explicit background offloading، isolated conformances برای main actor types. هنگام offloading — adopting Swift 6.2 concurrency با @concurrent یا resolving main-actor isolation استفاده کنید.
303	`swift-protocol-di-testing`	Protocol-based dependency injection برای testable Swift code — mock کردن file system، network و external APIs با استفاده از focused protocols و Swift Testing. هنگامی که Swift code نیاز به testing دارد و، file system یا network یا external APIs باید mocked شوند استفاده کنید.
304	`swiftui-patterns`	الگوهای معماری SwiftUI، state management با @Observable، view composition، navigation، performance و optimization و modern iOS/macOS UI best practices. هنگام ساخت یا بازبینی SwiftUI views، @Observable state، navigation یا render performance استفاده کنید.
305	`taste`	Creative-direction layer برای music videos و short-form edits در family angelcore/cloud-trance/hyperpop — یک named-genre aesthetic vocabulary، mood + color + light system، beat-synced editing grammar و یک pipeline chaining ECC's video skills از b-roll generation تا distribution. هنگامی که یک video باید intentional (نه merely functional) احساس شود — music videos، fancams، moodboard-driven reels یا giving AI-generated b-roll یک coherent visual direction استفاده کنید.
306	`taste-application`	Generate کردن new video علیه یک distilled style pack و cut کردن آن به یک plan — finished piece کردن takes از reference's cut rhythm، generate کردن روی fal، grade کردن با cut، pack's measured LUT at measured کردن cadence، weave کردن existing footage، composite کردن overlay plates، mint کردن 3D props و verify کردن

		result numerically. هنگامی که کاربر می‌خواهد video در یک captured style بسازد، existing footage supplement کند یا generated clips را به یک real edit assemble کند استفاده کنید.
307	`taste-distillation`	Measure کردن یک set of reference videos به یک reusable style pack — colour grade به‌عنوان 3D LUT، cut rhythm به‌عنوان shot-length distribution، hero stills، screen-blend overlay plates و text spec برای یک generative model. هنگامی که کاربر می‌خواهد look of reference footage را capture کند، یک repeatable look بسازد، assets از references mint کند یا someone's grade و pacing را reproduce کند استفاده کنید.
308	`tasteforge-video`	برای video، file-driven multimodal image و 3D-asset discovery؛ taste interviews؛ distill یا apply workflows؛ style-pack validation؛ editable EDL/FCPXML export؛ provenance audits؛ و offline planning که باید fail closed شود قبل از provider generation استفاده کنید.
309	`tdd-workflow`	گردش‌کار test-driven development: ابتدا یک failing test بنویسید، ببینید fail می‌شود، smallest change را برای green implement کنید، سپس refactor با coverage +٪۸۰ در integration، unit و E2E tests. هنگام نوشتن یک feature جدید، fixing یک bug، refactoring یا وقتی گفته می‌شود failing tests first بنویسید استفاده کنید.
310	`team-agent-orchestration`	Run کردن team-based orchestration برای agent squads: work items با owners و agent Kanban state، scope، branch isolation، control pane visibility و merge gates. هنگام هماهنگی multiple agents به‌صورت parallel در worktrees — multi-agent fan-out، agent Kanban، squad coordination یا branches merging agent output به یک product استفاده کنید.
311	`team-builder`	Interactive picker که available agent personas را از طریق claude agents command و agents/ markdown globs discover می‌کند، آنها را به domains group می‌کند، user را تا five انتخاب می‌کند، آنها را به‌صورت parallel روی یک task dispatch می‌کند و agreements و conflicts را به یک unified report synthesize می‌کند. هنگام composing یک team of agents، browsing available agent personas یا running several specialist agents in parallel استفاده کنید.
312	`terminal-opener`	Open کردن یک executable و argument array آن در یک visible terminal window از طریق یک reusable، shell-free launch plan با detached fallback، capability detection، JSON، dry-run و standalone recovery modes. هنگامی که Codex نیاز دارد یک sandbox، local development process، SSH session یا interactive CLI، other argv-based command را در یک new host terminal باز کند؛ diagnose کند آیا یک available terminal supported است؛ یا یک actionable plan فراهم کند وقتی requested terminal unsupported است استفاده کنید.
313	`terminal-ops`	گردش‌کار repo execution مبتنی بر evidence برای ECC. هنگامی که user می‌خواهد یک command run شود، یک repo checked شود، یک CI failure debug شود یا یک narrow fix pushed شود با exact proof of what was executed و verified استفاده کنید.
314	`test-collaboration`	Inventory و govern کردن test assets یک project. Connect کردن bugs، risks، business rules، requirements و cross-boundary contracts به stable TEST-IDs و executable evidence در repository's existing test registry یا TESTS.md. برای test inventory، test gap analysis، unit/integration/contract/E2E/smoke classification، bug-to-pre-delivery و test regression protection، consumer/provider contract testing، test-necessity review evidence review استفاده کنید.
315	`tinystruct-patterns`	راهنمای expert برای developing با tinystruct Java framework. هنگام کار روی tinystruct codebase یا هر project built on tinystruct — شامل generating bin/dispatcher و bin/dispatcher.cmd launcher scripts وقتی project lacks them، creating Application classes، @Action-mapped routes، unit tests، ActionRegistry، HTTP/CLI dual-mode handling، built-in HTTP server، event system، JSON با Builder/Builders، database persistence با AbstractData، POJO generation، Server-Sent Events (SSE)، file uploads و outbound HTTP networking استفاده کنید.
316	`token-budget-advisor`	ارائهٔ انتخاب (25%/50%/75%/100%) response depth با token estimates قبل از answering، سپس answer در آن level. هنگامی که user می‌خواهد response length یا token budget را control کند، مانند 'short'، 'token budget'، 'brief answer'، 'version' استفاده کنید. Skip اگر depth قبلاً در این session set شده یا 'token' به معنای auth/payment token است.
317	`token-card`	Render کردن یک token-usage stat card از local Claude Code، Codex، Gemini CLI و OpenCode session logs و commit کردن آن به repository به‌عنوان یک file. هنگامی که someone یک current usage figure در README می‌خواهد یا asks برای یک shareable card of coding-agent activity استفاده کنید.

318	`ui-demo`	Record کردن polished UI demo videos با استفاده از Playwright. هنگامی که user می‌خواهد یک demo، walkthrough، screen recording یا tutorial video of یک web application بسازد استفاده کنید. WebM videos با visible cursor، natural pacing و professional feel تولید می‌کند.
319	`ui-to-vue`	هنگامی که user UI screenshots یا design exports دارد که نیاز به batch conversion به Vue 3 components دارند، به‌ویژه با Vant، Element Plus یا Ant Design Vue استفاده کنید.
320	`uncloud`	هنگام managing یک Uncloud cluster — deploying services، configuring Caddy ingress، adding static proxy routes برای logs، inspecting، scaling، publishing ports، non-cluster devices یا managing machines و volumes با `uc` CLI` استفاده کنید.
321	`unified-memory`	Share کردن durable، inspectable context و handoffs بین OpenCode، Cursor، Hermes، Codex، Claude و سایر agents از طریق local ECC Memory Vault. هنگامی که یک agent باید work state save کند، context transfer کند، task agent دیگری را resume کند یا shared project knowledge را search کند استفاده کنید.
322	`unified-notifications-ops`	عملیات notifications به‌عنوان یک ECC-native workflow در hooks، desktop alerts، Linear، GitHub و connected communication surfaces. هنگامی که real problem alert routing، deduplication، escalation یا inbox collapse است استفاده کنید.
323	`verification-loop`	اجرای six-phase verification of work یک Claude Code session — build، type check، lint، tests با coverage، security grep و diff review — سپس تولید PASS/FAIL verification report. هنگام verifying work پس از completing یک feature یا refactor، قبل از creating PR یا وقتی quality gates باید pass شوند استفاده کنید.
324	`video-editing`	گردش‌کار AI-assisted video editing برای cutting، structuring و augmenting real footage. پوشش full pipeline از raw capture از طریق fal.ai، ElevenLabs، Remotion، FFmpeg و final polish در Descript یا CapCut. هنگامی که user می‌خواهد video edit کند، footage cut کند، vlogs بسازد یا video content build کند استفاده کنید.
325	`videodb`	Ingest، index، search، edit و monitor کردن video و audio با VideoDB Python SDK — upload از files، URLs یا RTSP feeds، build کردن spoken و scene indexes با timestamped search و transcode، playable clips و reframe، timeline edits (subtitles، overlays، dubbing) و run کردن real-time alerts روی live streams یا desktop capture. هنگام کار با streaming، transcoding، clipping، transcription، video search یا live video alerts استفاده کنید.
326	`viral-hooks`	Generate کردن ۳ short-form video scripts (Reels، TikTok، YouTube، برای scroll-stopping hook variant LinkedIn، X). هنگامی که user در حال نوشتن script برای short-form video است، hook ideas می‌خواهد، opening line برای Reel/post نیاز دارد یا script paste می‌کند و alternative hooks می‌خواهد استفاده کنید. از hook ۱۰۰ library در ۱۰ formula psychology trigger (curiosity، contrarian، authority، emotional، listicle، question، story، negation، specificity، confession) pull می‌کند.
327	`visa-doc-translate`	Translate کردن visa document images (JPG یا PNG) به English از طریق OCR و تولید bilingual PDF pairing original image با formatted certified-style translation. هنگامی که visa application نیاز به document translated to English، official certificate OCR'd و translated یا bilingual translation PDF برای immigration paperwork دارد استفاده کنید. HEIC، retirement certificates و bank deposit، employment، income؛
328	`vite-patterns`	الگوهای Vite build tool شامل config، plugins، HMR، env variables، proxy setup، SSR، library mode، dependency pre-bundling و build optimization. هنگام کار با Vite plugins، vite.config.ts یا Vite-based projects activate می‌شود.
329	`vue-patterns`	الگوهای Vue.js 3 Composition API، component architecture، reactivity best practices، Pinia state management، Vue Router navigation و Nuxt SSR patterns. برای Vue، Nuxt، Vite یا Pinia projects activate می‌شود. هنگام ساخت یا بازبینی Vue 3، Nuxt، reactivity یا Pinia code — Composition API یا router navigation استفاده کنید.
330	`windows-desktop-e2e`	E2E testing برای Windows native desktop apps (WPF، WinForms، Win32/MFC، Qt) با استفاده از pywinauto و Windows UI Automation. هنگام نوشتن E2E tests برای یک Windows native desktop app با pywinauto یا UI Automation استفاده کنید.
331	`workspace-surface-audit`	Audit کردن active repo، MCP servers، plugins، connectors، env surfaces و harness setup، سپس recommend highest-value ECC-native skills، hooks، agents و operator workflows. هنگامی که user می‌خواهد help

		setting up Claude Code یا understanding what capabilities واقعاً در environment او available هستند استفاده کنید.
332	`x-api`	X/Twitter API integration برای posting tweets، threads، reading timelines، search و analytics. پوشش OAuth و auth patterns، rate limits و platform-native content posting. هنگامی که user می‌خواهد programmatically با X interact کند استفاده کنید.
333	`x-audit`	Post-mortem روی آنچه کاربر قبلاً در X منتشر کرده — کدام postها واقعاً کار کردند، چرا و چه چیزی باید stop شود — از analytics export خود. هنگامی که user X analytics CSV یا past posts خود را share می‌کند و می‌پرسد 'what's working'، 'why did this flop'، 'read my analytics'، 'audit my account' یا می‌خواهد بداند links، threads یا posting time واقعاً matter می‌کنند استفاده کنید.
334	`x-comment`	نوشتن replies روی postها و threads دیگران در X که به‌عنوان یک person با something to add خوانده می‌شوند، نه reply guy. هنگامی که user post را paste می‌کند و reply می‌خواهد، می‌گوید 'comment on this'، 'reply to this'، 'what do I say here'، 'engage with this'، 'thread' یا batch replies برای daily engagement round می‌خواهد استفاده کنید.
335	`x-dm`	نوشتن X DMهایی که answer می‌گیرند — first message، دو follow-up — و triage کردن DM inbox و message requests کاربر. هنگامی که user می‌گوید 'how do I follow up'، 'outreach on X'، 'cold DM'، 'DM this person'، 'what do I say to this DM'، 'go through my message requests'' یا به someone specific در X reach out می‌کند استفاده کنید.
336	`x-human`	strip کردن machine fingerprint از هر draft — em dashes، AI slop words، invisible watermark characters و score کردن در برابر five-check detection panel قبل از ارسال. هر زمان text باید human به نظر برسد، هنگامی که user می‌گوید 'de-slop'، 'remove the em dashes'، 'does this sound like AI'، humanize، 'this sounds like ChatGPT'، 'this' یا قبل از هر post، thread، reply یا DM از این pack استفاده کنید.
337	`x-plan`	Plan کردن week کاربر روی X — چه چیزی post کنید، کی post کنید و با چه کسی engage کنید. هنگامی که user می‌گوید 'I have nothing to post'، 'posting schedule'، 'content calendar'، 'what should I post'، 'plan my week' یا 'about weekly mix of posts، threads و replies و list of accounts to engage می‌خواهد استفاده کنید.
338	`x-post`	نوشتن یک X (Twitter) post از یک raw idea با استفاده از ۲۱ hook formula، در voice خود کاربر، counted the way X counts characters و humanized so it does not read as AI. هر زمان کاربر 'post about X'، 'tweet، X post، hook، 'turn this into a tweet'، long post برای Premium یا hook options می‌خواهد استفاده کنید. سه scored hook، یک full draft و یک copy-ready block تولید می‌کند که هرگز بدون explicit yes publish نمی‌شود.
339	`x-profile`	امتیازدهی X profile از ۱۰۰ در برابر part rubric-۱۲ و rewrite کردن parts که points از دست می‌دهند — bio، pinned post، display name، header، link. هنگامی که user می‌گوید 'score my X bio'، 'fix my profile'، 'optimize my profile'، 'rewrite my bio'، 'what should I pin'، 'Twitter profile' یا profile خود را paste می‌کند و می‌پرسد چگونه خوانده می‌شود استفاده کنید.
340	`x-reply`	Handle کردن replies زیر postهای خود کاربر در sort — X کردن هر کدام، سپس draft کردن answers به ترتیب importance. هنگامی که user replies روی post خود را paste می‌کند، می‌گوید 'handle my replies'، 'reply to these'، 'someone said X under my post'، 'how do I answer this'' یا با critic، troll یا lead در mentions deal می‌کند استفاده کنید.
341	`x-repurpose`	تبدیل یک long asset — YouTube video، podcast، newsletter، blog post، transcript، LinkedIn post یا client call به یک week of X posts و یک thread. هنگامی که user می‌گوید 'I have a video/newsletter/transcript'، 'turn this into tweets'، 'repurpose this' یا چیزی طولانی paste می‌کند و می‌خواهد روی X باشد استفاده کنید.
342	`x-thread`	ساخت یک X thread: first post که whole thing را sell می‌کند، one idea per post، cut کردن برای fit X's weighted character count و last post که people act on. هنگامی که user می‌گوید 'turn this into a thread'، 'write a thread'، 'tweetstorm' یا ایدهٔ list-shaped یا step-shaped دارد که در یک post cramped می‌شود استفاده کنید.
343	`x-viral`	The virality engine. پیدا کردن postهایی که در niche کاربر روی X واقعاً outperforming هستند، rank کردن آنها بر اساس اینکه هر کدام چقدر account خود را beat کرده‌اند، break down کردن هر کدام به hook، structure و format، سپس recreate کردن winners در voice خود کاربر برای audience او. هنگامی که user می‌گوید 'find viral posts'، 'what's working on X right now'، 'what are people posting in my niche'، 'reverse engineer this account'،

		build me a swipe file'، 'why did this tweet blow up'، 'recreate this post for me'' یا has nothing to post و no evidence استفاده کنید.
344	`yt-audit`	Audit کردن یک YouTube channel end to end — packaging، consistency، first fifteen seconds و what to fix first. هنگامی که user می‌گوید 'review my videos'، 'why isn't my channel growing'، 'audit my channel' یا channel URL paste می‌کند استفاده کنید.
345	`yt-chapters`	نوشتن YouTube chapters از یک transcript، validated در برابر YouTube's own rules so they actually render. هنگامی که user می‌گوید 'break this video into sections'، 'timestamps'، 'add chapters' استفاده کنید.
346	`yt-comment`	Draft کردن replies به YouTube comments در voice creator، triaged by which ones are worth answering. هنگامی که user می‌گوید 'someone asked X'، 'handle the comment section'، 'reply to my comments' یا comment thread paste می‌کند استفاده کنید.
347	`yt-edit`	تبدیل یک transcript یک raw recording به یک edit decision list — dead air، filler cues و retakes، با timecodes. هنگامی که user می‌گوید 'I rambled'، 'tighten this video'، 'cut the dead space'، 'edit this' یا هر request برای shorten کردن footage از یک transcript استفاده کنید.
348	`yt-package`	نوشتن و lint کردن title و thumbnail برای یک YouTube video به‌عنوان یک pairing، checking truncation، duplication و vagueness قبل از publish. هنگامی که user می‌گوید 'what should I call this'، 'title ideas'، 'my CTR is bad'، 'thumbnail text' یا هر request برای rename یا repackage یک existing video استفاده کنید.
349	`yt-plan`	Plan کردن یک week یا month of YouTube uploads — چه چیزی post کنید، کی و در چه order، sized به capacity واقعی creator. هنگامی که user می‌گوید 'I have no idea what to make next'، 'what should I post'، 'content calendar'، 'plan my week' استفاده کنید.
350	`yt-retention`	خواندن یک YouTube Studio audience-retention export و یافتن جایی که viewers واقعاً leave می‌کنند، سپس گفتن چه چیزی باید change شود. هنگامی که user می‌گوید 'my retention is bad'، 'why do people stop watching' یا retention chart CSV paste می‌کند یا 'fix my pacing' استفاده کنید.
351	`yt-script`	نوشتن یک YouTube video script از یک raw idea — hook options از ۲۱، formula، scored، سپس full spoken script با retention beats marked. هر زمان کاربر 'write my next video script، hook، opening line، 'what should I say' video' می‌خواهد یا قصد record کردن دارد و first fifteen seconds را ندارد استفاده کنید.
352	`yt-seo`	نوشتن tags، description و search-facing text برای یک YouTube video، aimed at query که یک real person می‌نویسد. هنگامی که user می‌گوید 'nobody is finding this'، 'help this video get found'، 'SEO'، 'tags'، 'write my description' استفاده کنید.
353	`yt-shorts`	پیدا کردن Shortsهایی که در یک long video پنهان هستند و نوشتن آنها، با استفاده از transcript برای pick کردن self-contained moments. هنگامی که user می‌گوید 'what should I clip'، 'repurpose this video'، 'clip this'، 'cut this into shorts' استفاده کنید.
354	`yt-viral`	پیدا کردن آنچه در niche کاربر روی YouTube واقعاً کار می‌کند و rank کردن آن بر اساس اینکه هر video چقدر channel خود را beat کرده، سپس نام‌گذاری formula. هنگامی که user می‌گوید 'find viral videos'، 'what's working right now'، competitor research'، 'why did this blow up'، 'in my niche' یا swipe file می‌خواهد استفاده کنید.

۵. Tools (9)

ابزارهای سفارشی TypeScript در `/config/opencode/tools./~` . به‌صورت خودکار توسط opencode بارگذاری می‌شوند.

#	Tool	توضیحات کامل (فارسی)

1	`changed-files`	فهرست فایل‌های تغییریافته توسط agentها در این نشست به‌صورت درخت ناوبری. نشانگرهای افزودن (+)، تغییر (~) و حذف (-) را نمایش می‌دهد. برای مشاهدهٔ تغییرات استفاده کنید.
2	`check-coverage`	بررسی پوشش تست در برابر threshold و شناسایی فایل‌های با پوشش کم. گزارش‌های پوشش را از مکان‌های رایج می‌خواند.
3	`dependency-analyzer`	تحلیل وابستگی‌های پروژه برای بسته‌های قدیمی، آسیب‌پذیری‌های امنیتی و وابستگی‌های بلااستفاده. از npm، pnpm، yarn و bun پشتیبانی می‌کند.
4	`format-code`	تشخیص formatter برای فایل و بازگرداندن دستور دقیق اجرا (Biome، Prettier، Black، gofmt، rustfmt، swift-format). تولید دستورات cross-platform.
5	`git-summary`	تولید خلاصهٔ git با commit، status، branchهای اخیر و آمار diff اختیاری.
6	`index`	ابزار index برای ثبت و مدیریت ابزارها در opencode.
7	`lint-check`	تشخیص linter برای مسیر هدف و بازگرداندن دستور برای check/fix. تولید دستورات cross-platform.
8	`run-tests`	اجرای test suite با پوشش، watch mode یا patternهای تست خاص. package manager (npm، pnpm، yarn، bun) و test framework را به‌صورت خودکار تشخیص می‌دهد.
9	`security-audit`	اجرای audit امنیتی جامع شامل آسیب‌پذیری‌های وابستگی، اسکن secret و مشکلات امنیتی رایج.

۶. Plugins (4)

پلاگین‌های TypeScript در `~/.config/opencode/plugins/` .

#	Plugin	توضیحات کامل (فارسی)
1	`auto-markitdown`	پلاگین MarkItDown که فرمت‌های مختلف فایل (PDF، DOCX، XLSX، PPTX، HTML، CSV، JSON، XML، ZIP، EPUB، تصاویر، صدا، ویدیو، URLهای YouTube) را قبل از خواندن یا ویرایش در opencode به Markdown تبدیل می‌کند.
2	`index`	پلاگین اصلی ECC برای OpenCode v2 (framework-free). hookها را ثبت می‌کند: tool.execute.before/after، sniffing دستورات Bash، session.created/idle/deleted، file.watcher، shell.env، compaction، permission.ask، tool map. formatter و test auto-approvals را انجام می‌دهد.
3	`persian-typocorrect`	نرمال‌ساز متن فارسی که حروف عربی/فارسی، ارقام، ZWNJ، نقطه‌گذاری، فاصله‌گذاری و کشیده را اصلاح می‌کند. deterministic و idempotent. موارد ambiguous را به‌صورت خودکار تغییر نمی‌دهد.
4	`smart-fallback`	پلاگین smart-fallback که در صورت شکست model، به model جایگزین می‌رود. قابلیت‌های model (text، vision) را بررسی می‌کند.

۷. Hooks (27 hook در 8 گروه رویداد)

hook ها در `~/.config/opencode/hooks/hooks.json` و `hooks.metadata.json` تعریف شده‌اند. رویدادهای OpenCode را به اسکریپت‌های ECC متصل می‌کنند.

PreToolUse (10)

ID	توضیحات کامل (فارسی)
`pre:bash:dispatcher`	dispatcher یکپارچهٔ Bash preflight برای quality، tmux، push و GateGuard checks.
`pre:powershell:gateguard-fact-force`	Gate قدرت PowerShell: بررسی دستورات destructive بدون اجرای hookهای Bash-only.
`pre:write:doc-file-warning`	هشدار فایل doc: هشدار دربارهٔ فایل‌های مستندات غیراستاندارد (exit 0، فقط هشدار).
`pre:edit-write:suggest-compact`	پیشنهاد compaction دستی در فواصل منطقی.
`pre:observe:continuous-learning`	ثبت مشاهدات tool use برای continuous learning.
`pre:governance-capture`	ثبت رویدادهای governance (secrets، نقض policy، درخواست approval). با ECC_GOVERNANCE_CAPTURE=1 فعال می‌شود.
`pre:config-protection`	مسدود کردن تغییرات فایل‌های config linter/formatter. agent را به رفع کد هدایت می‌کند.
`pre:mcp-health-check`	بررسی سلامت MCP server قبل از اجرای MCP tool و مسدود کردن تماس‌های ناسالم.
`pre:edit-write:gateguard-fact-force`	Gate fact-forcing: اولین Edit/Write/MultiEdit/NotebookEdit هر فایل را با سؤالات مبتنی بر کلاس target deny می‌کند. targetهای حساس (secrets، keys، auth/payment code، migrations، CI workflows) و فایل‌های hard-linked همیشه gate می‌شوند.
`pre:hookify-runtime`	ارزیابی Hookify rules پروژه-محلی قبل از اجرای Bash، PowerShell و file tools.

UserPromptSubmit (1)

ID	توضیحات کامل (فارسی)
`prompt:hookify-runtime`	ارزیابی Hookify prompt rules پروژه-محلی قبل از پردازش prompt کاربر.

PreCompact (1)

ID	توضیحات کامل (فارسی)
`pre:compact`	ذخیرهٔ state قبل از context compaction.

SessionStart (2)

ID	توضیحات کامل (فارسی)
`session:start`	بارگذاری context قبلی و تشخیص package manager در نشست جدید.
`session-start:plan-canvas-sessions`	نمایش sessionهای باز Plan Canvas برای resume.

PostToolUse (2)

ID	توضیحات کامل (فارسی)
`post:dispatcher:sync`	اجرای hookهای synchronous PostToolUse در یک process.
`post:dispatcher:async`	اجرای hookهای asynchronous PostToolUse در یک process.

PostToolUseFailure (2)

ID	توضیحات کامل (فارسی)
`post:mcp-health-check`	ردیابی تماس‌های ناموفق MCP، علامت‌گذاری serverهای ناسالم و تلاش reconnect.
`post:skill:track`	ثبت شکست‌های سخت ابزار Skill برای telemetry سلامت skill.

Stop (8)

ID	توضیحات کامل (فارسی)
`stop:hookify-runtime`	ارزیابی Hookify completion rules قبل از توقف Claude.
`stop:plan-canvas-pending`	ارسال feedbackهای ناگفتهٔ Plan Canvas قبل از توقف.
`stop:format-typecheck`	فرمت (Biome/Prettier) و typecheck (tsc) batch همهٔ فایل‌های JS/TS ویرایش‌شده در این response — یک‌بار در Stop اجرا می‌شود.
`stop:check-console-log`	بررسی console.log در فایل‌های تغییریافته پس از هر response.
`stop:session-end`	ذخیرهٔ session state پس از هر response (Stop transcript_path را حمل می‌کند).
`stop:evaluate-session`	ارزیابی نشست برای الگوهای قابل استخراج.
`stop:cost-tracker`	ردیابی متریک‌های token و cost به ازای هر session.
`stop:desktop-notify`	ارسال notification desktop (macOS/WSL) با خلاصهٔ task پس از پاسخ Claude.

SessionEnd (1)

ID	توضیحات کامل (فارسی)
`session:end:marker`	نشانگر چرخهٔ پایان session (non-blocking).

۸. MCP Servers (34 تعریف‌شده)

تعریف‌شده در `config/opencode/mcp-configs/mcp-servers.json./~` . باید به‌صورت دستی در `claude.json./~` یا opencode config فعال شوند. `YOUR_*_HERE` Placeholders را با مقادیر واقعی جایگزین کنید.

#	Server	توضیحات کامل (فارسی)
1	`browser-use`	agent browser AI برای taskهای web.
2	`browserbase`	sessionهای browser ابری از طریق Browserbase.
3	`clickhouse`	کوئری‌های ClickHouse analytics.
4	`cloudflare-docs`	جستجوی مستندات Cloudflare.

5	`cloudflare-observability`	observability/logs Cloudflare.
6	`cloudflare-workers-bindings`	bindingsهای Cloudflare Workers.
7	`cloudflare-workers-builds`	buildهای Cloudflare Workers.
8	`codescene`	CodeScene Code Health MCP (اختیاری).
9	`confluence`	یکپارچه‌سازی Confluence Cloud — جستجوی page، بازیابی محتوا، بررسی spaceها.
10	`context7`	جستجوی مستندات زنده — با /docs و skill documentation-lookup استفاده می‌شود.
11	`devfleet`	هماهنگی dispatch agent — multi-agentهای Claude Code موازی در worktreeهای isolated.
12	`ecc-memory-vault`	Memory Vault ECC محلی مشترک بین Claude، Codex، Hermes، Cursor، OpenCode. جستجوی memory پروژه+تیم.
13	`evalview`	تست snapshot — regression agent AI رفتار، تشخیص regression در tool calls و کیفیت output.
14	`exa-web-search`	جستجوی web، research و ingestion داده از طریق Exa API.
15	`fal-ai`	تولید تصویر/ویدیو/صدا از طریق modelهای fal.ai.
16	`filesystem`	عملیات filesystem (مسیر خود را تنظیم کنید).
17	`firecrawl`	Web scraping و crawling.
18	`github`	عملیات GitHub — PRs، issues، repos.
19	`ito-compute`	MCP اختیاری Itô compute. ابزارهای ito_accept، ito_status، ito_find، ito_auth را ارائه می‌دهد.
20	`jira`	یکپارچه‌سازی Jira — جستجو، ایجاد، به‌روزرسانی، transition issue، commentها.
21	`laraplugins`	کشف pluginهای Laravel.
22	`longhand`	تاریخچهٔ lossless sessionهای Claude Code — ایندکس thinking blocks، file edits، tool calls.
23	`magic`	کامپوننت‌های Magic UI.
24	`memory`	memory پایدار بین sessionها.
25	`memxus`	memory پایدار universal بین ابزارهای AI.
26	`nexus`	proxy محلی cost/privacy — پرس‌وجوی usage و savings خودتان، مسیریابی به ارزان‌ترین model و mask کردن secrets/PII قبل از egress.
27	`omega-memory`	memory agent با جستجوی semantic، هماهنگی multi-agent و knowledge graphs.
28	`parallel-search`	جستجوی web موازی بهینه‌شده برای LLM.
29	`playwright`	اتوماسیون browser و تست از طریق Playwright.
30	`railway`	deployments Railway.
31	`sequential-thinking`	استدلال chain-of-thought.

32	`supabase`	عملیات پایگاه دادهٔ Supabase.
33	`token-optimizer`	بهینه‌سازی token برای کاهش ۹۵٪+ context از طریق deduplication و compression.
34	`vercel`	deployments و پروژه‌های Vercel.

۹. روش استفادهٔ کلی

- **Agents:** با `agent-name@` در چت یا از طریق Task tool. مثال: `@tdd-guide`، `@code-reviewer`، `planner`.
- **Commands:** با `command-name/` در TUI. مثال: `/verify`، `/code-review`، `/tdd`، `plan`.
- **Skills:** با ابزار `skill` یا بارگذاری خودکار. مثال: `skill({ name: "tdd-workflow" })`.
- **Tools:** به‌صورت خودکار در دسترس agent. مثال: `check-coverage`، `security-audit`، `run-tests`.
- **Plugins:** به‌صورت خودکار بارگذاری می‌شوند (index.ts hook ها را ثبت می‌کند).
- **Hooks:** به‌صورت خودکار بر اساس رویدادها (Stop، PostToolUse، PreToolUse و...) اجرا می‌شوند.
- **MCP:** نیاز به فعال‌سازی دستی و تنظیم API keys.

تولیدشده توسط ECC 2.2.3 | opencode | توضیحات به زبان فارسی